

WATCHDOG

עידו מאימן

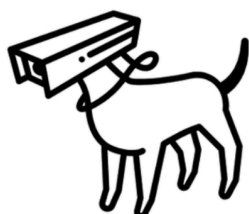
215825894

מורה מנחה: עידן פלדברג

פרויקט בחלופה הגנת סייבר ומערכות

הפעלה

תאריך הגשה: 24.5.2025



WATCHDOG

2	מבוא
2	יזום
4	פירוט תיאור המערכת
7	תיאור תחום הידע
7	פירוט מעמיק של היכולות שהוצגו בשלב הקודם ומעבר
10	מבנה / ארכיטקטורה
10	תיאור הארכיטקטורה של המערכת המוצעת
10	תיאור הטכנולוגיה הרלוונטית
11	תיאור זרימת המידע במערכת
13	תיאור האלגוריתמים המרכזיים בפרויקט
16	תיאור סביבת הפיתוח
17	תיאור פרוטוקול התקשורת
19	תיאור מסכי המערכת
22	תיאור מבני הנתונים
23	סקירת חולשות ואיומים
30	מימוש הפרויקט
30	חלק א'
44	חלק ב'
48	חלק ג'
51	מדריך למשתמש
51	עץ קבצים
51	התקנת המערכת
52	אופן הפעלת המערכת
68	רפלקציה / סיכום אישי
69	ביבליוגרפיה
70	נספחים

מבוא

ייזום

תיאור ראשוני של המערכת

WATCHDOG היא מערכת לניהול מצלמות אבטחה המבוססת על שידור וידאו בזמן אמת בין שני סוגי לקוחות: מצלמות ומשתמשים. המערכת מאפשרת למצלמות להעביר שידור חי, ולמשתמשים לצפות בשידורים אלו, ולשלוח התרעות בזמן אמת לגבי המתרחש במצלמה. בחרתי בפרויקט לאחר התייעצות עם המורה המנחה. לאחר שנתקלנו ברעיון של משקפי מציאות רבודה שמתריעים על סכנות בזמן אמת, החלטתי לממש רעיון דומה במערכת מצלמות אבטחה שמשתמשים אשר צופים במצלמות יוכלו להיות אלו שמודיעים על הסכנות ופוטנציאלית גם שימוש בבינה מלאכותית בשביל זיהוי סכנות אוטומטי. המוצר הסופי אמור לספק מערכת מבוססת רשת המאפשרת חיבור בין מצלמות למשתמשים בצורה מאובטחת, פשוטה להפעלה, ומותאמת לשימוש אישי, ביתי או משרדי.

הגדרת הלקוח

המערכת מיועדת בעיקר לשימוש פרטי או לעסקים קטנים. המערכת מהווה תחליף למערכות יקרות ומסובכות שבהם צריך להגדיר מצלמות אבטחה ייעודיות, במערכת זו ניתן להשתמש במצלמות רשת פשוטות שרק צריך לחבר למחשב עם גישה לרשת.

הגדרת יעדים ומטרות

המטרות המרכזיות של מערכת WATCHDOG:

- לאפשר חיבור בין מצלמות למשתמשים דרך תקשורת עם שרת.
- לאפשר למצלמה לשדר וידאו חי ולמשתמש לצפות בו דרך הממשק.
- לאפשר למשתמשים לשלוט במעבר בין מצלמות, צילום תמונות, הרשמה למצלמות ושידור התרעות.
- לשלוח התרעות אוטומטיות למשתמשים במקרה של אירוע.

בעיות, תועלות וחסכוניות

הבעיות שאנחנו מנסים לפתור:

- חוסר נוחות בשימוש במערכות אבטחה קלאסיות שמצריכות התקנות מורכבות וציוד יקר.
- חוסר באינטגרציה עם שירותי הודעות מידיים לשליחת התרעות.

- חשש מפני סכנות קרבות שניתן היה בקלות לזהותן.

תועלות המערכת:

- מערכת פשוטה לתפעול עם ממשק נוח.
- חיסכון כספי בהשוואה למערכות מסחריות.
- קבלת התרעות מיידיות ממקור אמין דרך טלגרם.

סקירת פתרונות קיימים

קיימות מערכות רבות כמו Arlo, Ring, או Google Nest. פתרונות אלה דורשים חומרה מיוחדת, התקנות מסובכות, ולעיתים קרובות יש צורך בביצוע תשלום חודשי עבור שירותי ענן. הפתרון שמציעה מערכת WATCHDOG שונה בכך שהוא מבוסס על מחשב אישי ותקשורת ישירה בין שרת ללקוחות, הוא לא תלוי בשירותי ענן חיצוניים וניתן להרחבה ושדרוג לפי הצורך.

סקירת טכנולוגיית הפרויקט

המערכת מבוססת על שפת פייתון, תוך שימוש בטכנולוגיות כגון: סוקטים, OpenCV, python-telegram-bot, CustomTkinter. לא מדובר בטכנולוגיות חדשות, אך השילוב ביניהן במערכת אחת הוא ייחודי. מפני שזו מערכת יותר ביתית מאשר מסחרית, קיים קושי בניהול המערכת עם מספר רב מאוד של לקוחות.

תיחום הפרויקט

הפרויקט עוסק בתחומים הבאים:

- רשתות
- מערכות הפעלה
- אבטחה
- הצפנה

הפרויקט אינו כולל:

- פתרונות ענן או הקלטה/אחסון ארוך טווח של וידאו.
- יכולת ניתוח תמונה מתקדם. למרות הרצון לשלב בינה מלאכותית לזיהוי סכנות בזמן אמת בפרויקט, עקב מגבלות הזמן שהוצבו בפניי לא התאפשר מימוש רעיון זה בזמן.

פירוט תיאור המערכת

המערכת שפותחה היא מערכת אבטחה חכמה הכוללת שרת מרכזי, ממשק משתמש וממשק מצלמה. המערכת מאפשרת למשתמשים להירשם, להתחבר, לצפות בשידור חי ממצלמות האבטחה, ולקבל התראות דרך טלגרם כאשר מתרחש אירוע חריג. מצלמות האבטחה נרשמות גם הן למערכת, משדרות נתונים לשרת המרכזי אשר מנהל את ההרשאות, מקבל את השידורים ומפיץ אותם למשתמשים הבוחרים לצפות במצלמה זו. המערכת משתמשת בהצפנה מלאה מהסוגים RSA ו-AES לצורך הגנה על פרטים רגישים ומכילה מנגנון אימות דו-שלבי באמצעות קוד אימות הנשלח דרך טלגרם.

יכולות המערכת

- הרשמה למערכת
- התחברות למערכת
- שיקוף וידאו מצד המצלמה
- הסתרת הוידאו מצד המצלמה
- החלפת מצלמה
- צילום מסך
- הרשמה/ביטול הרשמה למצלמה
- התרעת רשומים למצלמה
- שיקוף וידאו מצד המשתמש

בדיקות מתוכננות (קופסה שחורה)

- **בדיקת התחברות משתמש תקין**
נבדקת האפשרות של משתמש קיים להתחבר באמצעות שם משתמש וסיסמה תקינים. הצלחה תוביל לכניסה למערכת.
- **בדיקת התחברות עם סיסמה שגויה**
נבדקת המערכת מול מקרה של שלוש ניסיונות התחברות כושלים. המערכת אמורה לחסום את המשתמש ולהחזיר שגיאה מתאימה.
- **בדיקת הרשמה וקבלת קוד אימות**
בעת ההרשמה נשלח קוד אימות לטלגרם של המשתמש. נבדקת תקינות שליחת ההודעה והקבלה שלה.
- **בדיקת התחברות מצלמה לשרת**
מצלמה שמתחברת אמורה להעביר שידור חי לשרת. נבדקת יכולת הקבלה של השרת את התמונה.
- **בדיקת צפייה בשידור חי**
משתמש שמחובר וצופה במצלמה אמור לראות תמונה עדכנית. נבדקת הצגת השידור בממשק המשתמש.
- **בדיקת הרשמה עם שם משתמש קיים**
נבדקת פעולת ההרשמה כאשר מוזן שם משתמש שכבר קיים במסד הנתונים. המערכת אמורה להחזיר שגיאה ולא לאפשר הרשמה כפולה.
- **בדיקת הרשמה עם קוד אימות שגוי**
נבדק מקרה בו המשתמש מקליד קוד אימות שגוי לאחר ההרשמה. ההרשמה לא אמורה להתבצע ללא קוד נכון.
- **בדיקת התנתקות הלקוח מהשרת**
נבדקת פעולה של התנתקות יזומה מצד הלקוח. המערכת אמורה לטפל נכון בניתוק ולסגור את החיבור בשרת.

תכנון וניהול לוח זמנים לפיתוח

שלב	תיאור נוסף	לוח מתוכנן	לוח בפועל
בחירת רעיון לפרויקט והצעתו	בחירת הנושא לפרויקט שעליו אעבוד בחודשים לאחר מכן	שבוע	שבועיים
שרת עם תקשורת	פיתוח גרסה ראשונית של שרת אליו יוכלו להתחבר מספר לקוחות בו זמנית לגישה למערכת	יום	חמישה ימים
בסיס נתונים	פיתוח בסיס הנתונים לפרויקט עם כל הטבלאות והקשרים השונים ביניהן	שבוע	יומיים
העברת וידאו בין לקוחות בשרת	מציאת דרך להעברת וידאו בתקשורת ומימוש בפרויקט כנדרש	יום	שבוע
הוספת הצפנה	הוספת הצפנה לכל תחומי הפרויקט, RSA בכלליות, AES לתמונות הוידאו ו-Hash לסיסמאות בבסיס נתונים	יומיים	שלושה שבועות
תהליכי הרשמה והתחברות מסודרים למערכת	הוספת מערכת הרשמה והתחברות למערכת לאחר התחברות לשרת	שבוע	יום
אינטגרציה עם טלגרם	הוספת תמיכה לשימוש בטלגרם לניהול הודעות עם המשתמש ביצירת המשתמש ולהתרעות	יום	שבועיים
טיוב כללי של המערכת	שיפור אלגוריתמיקה לקנייה בקוד וייעול כללי	שבוע	שלושה שבועות
הוספת פיצ'רים	הוספת מגוון פיצ'רים למערכת כמו לקיחת תמונה ושיקוף הוידאו	יום	שבוע
עיצוב ממשק משתמש	לשנות את עיצוב ממשק המשתמש מעיצוב ברירת המחדל של CustomTkinter	שבוע	שלושה ימים
שילוב בינה מלאכותית	שימוש בספריית בינה מלאכותית חיצונית לזהות סכנות בזמן אמת	לאחר ביצוע כל השלבים הקודמים	לא יצא לפועל במסגרת הזמן

		מוידאו המצלמה ולתריע על כך אוטומטית	
--	--	---	--

ניהול הסיכונים בפרויקט והדרכים להתמודדות

תקלה בתקשורת בין לקוח לשרת:

- סיכון: ניתוק פתאומי או כשל בשליחת נתונים.
- דרך התמודדות: שימוש במעטפת try except לטיפול בשגיאות תקשורת.
- בפועל: נוספה התמודדות עם שגיאות כמו ConnectionResetError, socket.timeout, כולל סגירה מסודרת של החיבור.

עומס על השרת כתוצאה מריבוי לקוחות:

- סיכון: קריסת השרת או האטה חמורה.
- דרך התמודדות: שימוש בפונקציית listen בשביל להגביל את כמות הלקוחות שיוכלו להתחבר.
- בפועל: נוספה בקרה לסיום חיבורים וניתוקים בצורה מסודרת ובקרה בעת התחברות הלקוח לשרת אם יש מקום בשרת.

שמירה לא מוצפנת של נתונים רגישים:

- סיכון: חשיפת מידע רגיש בתקשורת.
- דרך התמודדות: שימוש בהצפנת RSA בכל העברת נתונים.
- בפועל: בעת העברת תמונות המצלמה היה צורך בשימוש ב-AES שיותר מהיר מ-RSA ולהצפנת הסימאות בבסיס הנתונים נעשה שימוש ב-Hash.

תיאור תחום הידע

פירוט מעמיק של היכולות שהוצגו בשלב הקודם ומעבר

יכולות כלליות

הרשמה למערכת

- **מהות היכולת:** רישום חשבון חדש במערכת על ידי הזנת פרטי הרשמה ואימותם.
- **אוסף יכולות/פעולות נדרשות:**
 - ממשק משתמש להזנת פרטים
 - קליטת נתונים
 - הצפנה ופענוח של הנתונים
 - תקשורת בין לקוח לשרת
 - בדיקה מול בסיס הנתונים האם החשבון כבר קיים
 - הכנסת פרטי החשבון לבסיס הנתונים

אובייקטים נחוצים: ממשק משתמש, הצפנה/פענוח, בסיס נתונים, תקשורת, מערכת הפעלה

התחברות למערכת

- **מהות היכולת:** התחברות משתמש או מצלמה לחשבון קיים במערכת.
- **אוסף יכולות/פעולות נדרשות:**
 - ממשק משתמש להזנת שם משתמש וסיסמה
 - הצפנה ושליחה לשרת
 - אימות נתונים מול בסיס הנתונים
 - אישור או סירוב התחברות
- **אובייקטים נחוצים:** ממשק משתמש, הצפנה/פענוח, בסיס נתונים, תקשורת, מערכת הפעלה

יכולות בצד חשבון מצלמה

שיקוף וידאו

- **מהות היכולת:** היפוך (שיקוף) הוידאו שהמצלמה כבר משדרת לכל מי שצופה בה.
- **אוסף יכולות/פעולות נדרשות:**
 - פתיחת מצלמת רשת והקלטת וידאו
 - קידוד ושליחת פריימים לשרת
- **אובייקטים נחוצים:** מצלמה, מודול OpenCV, מודול numpy .

הצגת/ביטול הצגת וידאו מצד המצלמה

- **מהות היכולת:** להציג את הוידאו שהמצלמה לוכדת למפעיל חשבון המצלמה, לאפשר לו לבטל את הצגת הוידאו לעצמו.
- **אוסף יכולות/פעולות נדרשות:**
 - עדכון ממשק המשתמש
- **אובייקטים נחוצים:** מצלמה, ממשק משתמש, מודול OpenCV

יכולות בצד חשבון משתמש

החלפת מצלמה לצפייה

- **מהות היכולת:** מעבר בין מצלמות שונות שהמשתמש רשום אליהן.
- **אוסף יכולות/פעולות נדרשות:**
 - שליחת בקשה לצפייה במצלמה חדשה
 - קבלת הוידאו מהשרת והצגתו
 - קבלת פרטים נוספים מהשרת על המצלמה אליה החליף והצגתם
- **אובייקטים נחוצים:** ממשק משתמש, תקשורת, שרת, הצפנה/פענוח.

צילום מסך

- **מהות היכולת:** שמירת תמונה מתוך סרטון המצלמה שמוצג למשתמש.
- **אוסף יכולות/פעולות נדרשות:**
 - שמירת התמונה לקובץ מקומי
 - ניהול תיקיית תמונות
- **אובייקטים נחוצים:** ממשק משתמש, מערכת קבצים, מערכת הפעלה

הרשמה/ביטול הרשמה למצלמה

- **מהות היכולת:** משתמש נרשם או מבטל את ההרשמה למצלמה מסוימת, כדי לקבל או לבטל את ההתרעות ממנה.
- **אוסף יכולות/פעולות נדרשות:**
 - שליחת בקשה לשרת להירשם/לבטל
 - עדכון סטטוס בבסיס הנתונים
- **אובייקטים נחוצים:** ממשק משתמש, תקשורת, בסיס נתונים, הצפנה/פענוח

שליחת התרעה

- **מהות היכולת:** שליחת התרעה לכל המשתמשים הרשומים למצלמה שצופה בה המשתמש דרך טלגרם.

- **אוסף יכולות/פעולות נדרשות:**

- ממשק כפתור התרעה
- שליחת בקשה לשרת
- שליפת המשתמשים הרשומים מבסיס הנתונים
- שליחת הודעה דרך בוט טלגרם

- **אובייקטים נחוצים:** ממשק משתמש, תקשורת, בסיס נתונים, טוקן טלגרם, API של Telegram, הצפנה/פענוח, HTTP.

שיקוף וידאו (מצד המשתמש בלבד)

- **מהות היכולת:** שיקוף הוידאו המתקבל מהמצלמה בצורה ישירה בחלון הצפייה של המשתמש.

- **אוסף יכולות/פעולות נדרשות:**

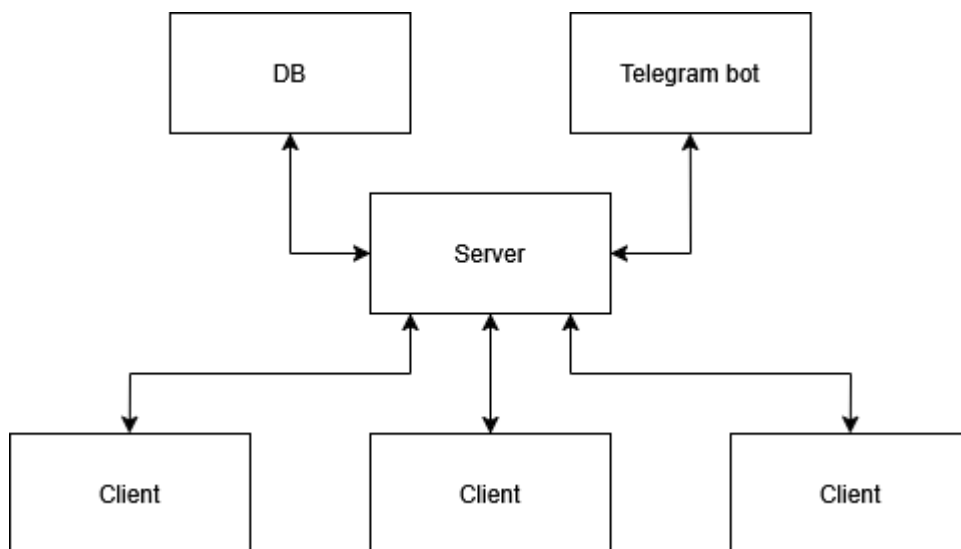
- קבלת זרם הוידאו מהשרת
- עיבוד והצגה בזמן אמת

- **אובייקטים נחוצים:** ממשק משתמש, תקשורת, numpy, הצפנה/פענוח

מבנה / ארכיטקטורה

תיאור הארכיטקטורה של המערכת המוצעת

הפרויקט בנוי משרת בעל גישה לבסיס נתונים ובוט טלגרם, ולקוחות. כל הלקוחות יכולים לתקשר אך ורק אם השרת שבעת הצורך ניגש לבסיס הנתונים ולבוט הטלגרם בשביל לבצע פעולות שונות כנדרש ממנו. (בדוגמה זו מוצגת תקשורת של השרת עם 3 לקוחות שונים)



תיאור הטכנולוגיה הרלוונטית

שפת תכנות:

הפרויקט מיושם בשפת התכנות פייתון, שהיא שפת תכנות רבת עוצמה, פשוטה לקריאה ולכתיבה, עם ספריות רבות המאפשרות פיתוח מהיר של יישומים מורכבים.

מערכת הפעלה:

המערכת פותחה ופועלת על מערכת ההפעלה חלונות (Windows).

תקשורת:

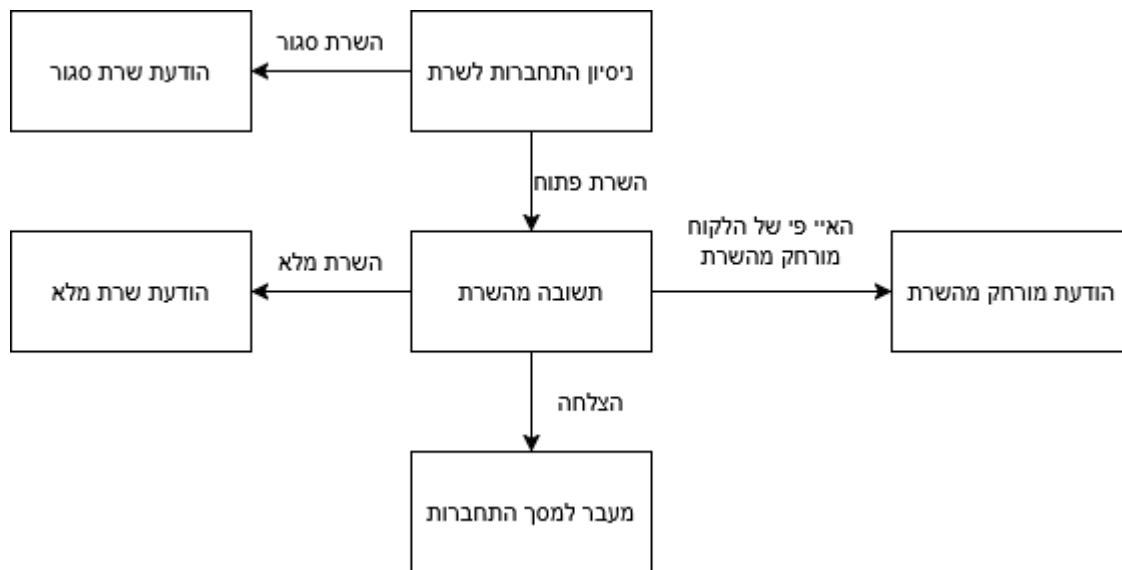
המערכת מבוססת על תקשורת TCP socket, שמאפשרת חיבור יציב ובטוח בין השרת ללקוחות. על מנת להגן על פרטיות המידע המועבר, התקשורת מוצפנת באמצעות אלגוריתמים קריפטוגרפיים כגון RSA ו-AES. בנוסף, נעשה שימוש ב-API של Telegram לצורך שליחת הודעות אימות והודעות מערכת למשתמשים, דרך פרוטוקול HTTP requests.

תחומי עניין:

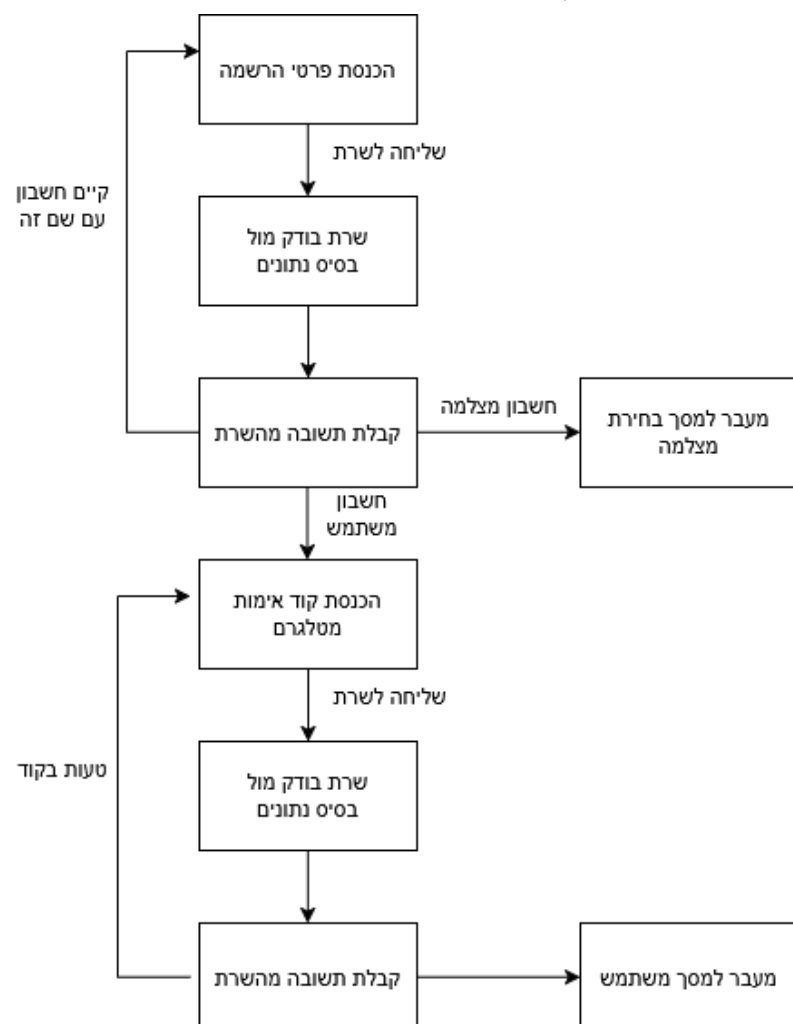
הפרויקט משלב תחומים של אבטחת מידע והצפנה, פיתוח מערכות ותקשורת רשת, ממשקי משתמש גרפיים (GUI) באמצעות CustomTkinter, וניהול מסדי נתונים באמצעות SQLite3. המערכת מיועדת למטרות אבטחה עם אפשרות לניהול משתמשים, מצלמות, והתרעות בזמן אמת.

תיאור זרימת המידע במערכת

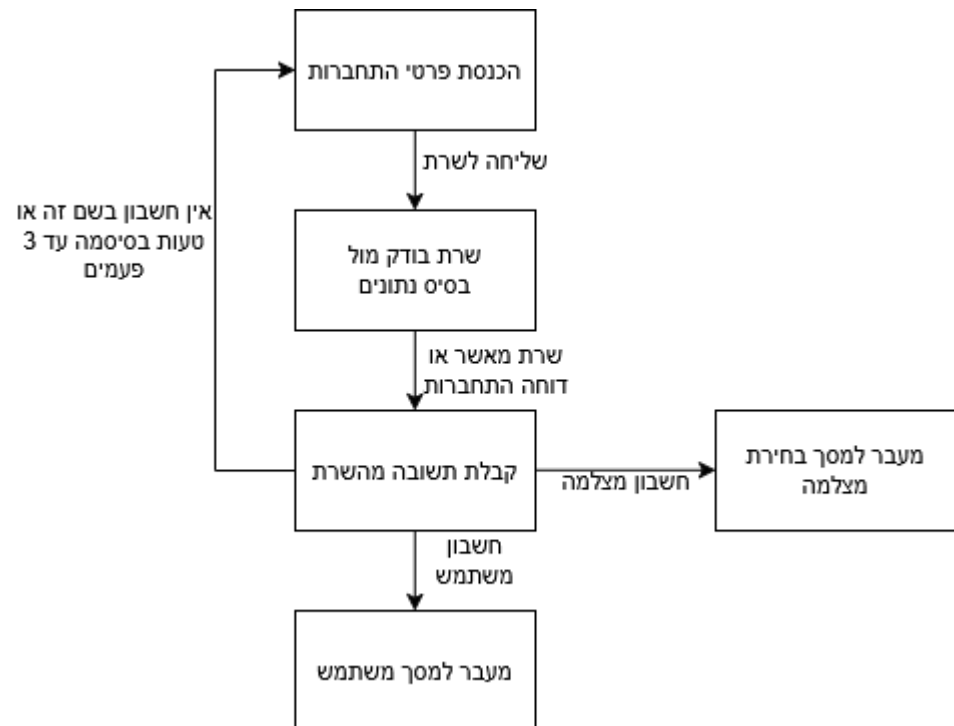
- התחברות לשרת:



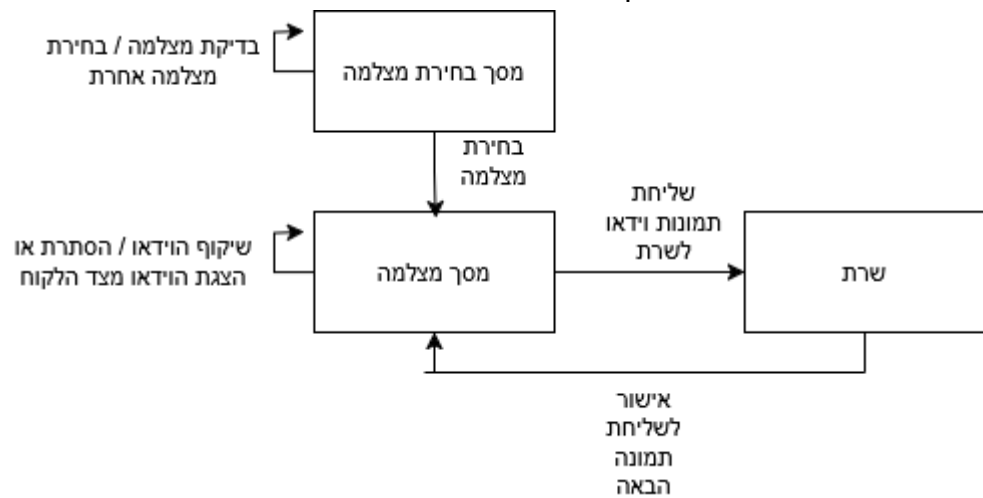
- הרשמה למערכת:



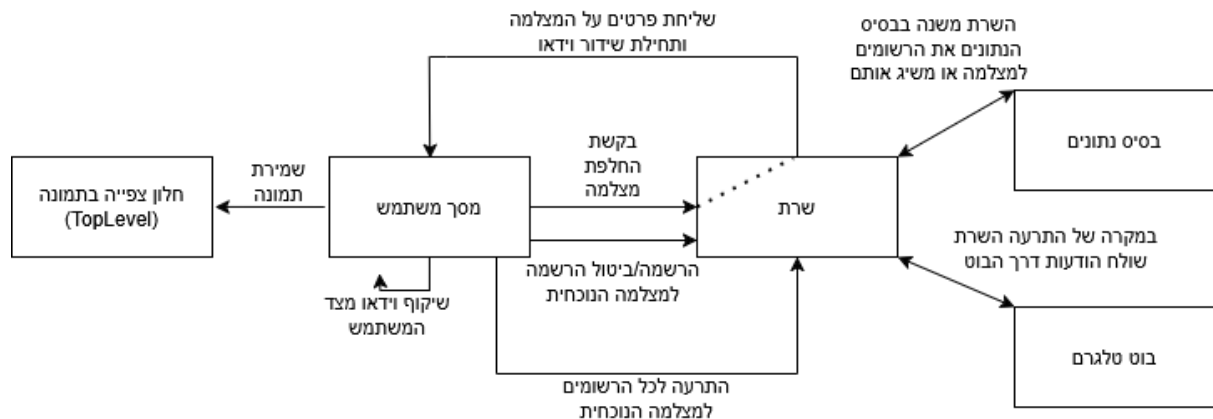
• התחברות למערכת:



• שימוש במערכת בחשבון מצלמה:



• שימוש במערכת בחשבון משתמש:



תיאור האלגוריתמים המרכזיים בפרויקט

אלגוריתם שליחת וקבלת תמונות בזמן אמת בין מצלמה למשתמש

המערכת נדרשת לאפשר שידור וידאו בזמן אמת מהמצלמה למשתמש דרך השרת. הבעיה האלגוריתמית המרכזית היא כיצד לבצע שידור זה בצורה מאובטחת, יעילה ומסונכרנת. השידור מתבצע בין שני סוגי לקוחות (מצלמה ומשתמש) דרך שרת מרכזי.

תיאור אלגוריתמים קיימים לפתרון הבעיה

1. שידור וידאו כפי שהוא
שליחת כל פריים כתמונת RGB ללא דחיסה או הצפנה. חסרון: צורך רוחב פס גבוה מאוד, לא מאובטח כלל.
2. שימוש בפרוטוקולים ייעודיים (כגון RTSP)
פרוטוקולים סטנדרטיים לשידור וידאו הכוללים מנגנוני דחיסה, סנכרון והצפנה. יתרון: אמינות גבוהה. חסרון: דורש ידע נוסף ותמיכה טכנית מיוחדת.
3. שידור מקודד ומוצפן ידנית
פתרון שבו כל פריים מומר לשחור-לבן, נדחס לפורמט JPEG, מוצפן בעזרת AES, ונשלח דרך סוקט TCP. קיים סנכרון בין שליחה לקבלה.

הפנייה למקורות רלוונטיים

- שימוש ב-TCP עם מודול סוקטים של פייתון: <https://docs.python.org/3/library/socket.html>
- מודול struct - אריזת נתונים באורך קבוע: <https://docs.python.org/3/library/struct.html>
- AES להצפנה: <https://cryptography.io/en/latest/hazmat/primitives/symmetric-encryption/>

סקירת הפתרון הנבחר

הפתרון שנבחר בפרויקט הוא שידור תמונות בפורמט JPEG דחוס ומוצפן דרך סוקט TCP. התהליך כולל את השלבים הבאים:

1. המרת התמונה לגווני אפור – בשביל אפקט שחור לבן המצוי לרוב במצלמות אבטחה.
2. דחיסה לפורמט JPEG – חיסכון בנפח (באיכות של 30%).
3. הצפנה ב-AES – הגנה על התמונה מפני יירוט.
4. שליחה דרך TCP – עם שליחה מוקדמת של גודל ההודעה עם ספרייט struct.
5. אישור קבלה מהצד המקבל – לפני שליחת הפריים הבא.

בצד המקבל, המשתמש מקבל את הנתונים, מפענח את הגודל, קורא את ההודעה המוצפנת המלאה, מפענח אותה בעזרת AES, ולאחר מכן מפענח את התמונה ומציג אותה.

נימוק הבחירה

- התאמה מדויקת לצרכי הפרויקט: פתרון מותאם אישית שעובד עם שאר חלקי המערכת.
- שליטה מלאה בתהליך.
- פשטות יחסית בהשוואה לאינטגרציה של פרוטוקולים מורכבים.
- ביצועים טובים באמצעות דחיסת JPEG והצפנת AES יעילה.

שלילת פתרונות חלופיים

- שידור וידאו כפי שהוא – צורך משאבים עצומים ואינו מאובטח, לא מעשי בסביבה רשתית טיפוסית.
- פרוטוקולים כמו RTSP – פתרונות מורכבים מדי לפרויקט זה, אשר דורשים ידע, ספריות, וקונפיגורציה מורכבת.
- UDP במקום TCP – היה עשוי לשפר ביצועים, אך כרוך באובדן מידע, לכן נבחר TCP לצורך אמינות מוחלטת בפריימים.

אלגוריתם שליחת הודעות בטלגרם

במערכת WATCHDOG, קיימת דרישה לשלוח התראות מיידיות לכל המשתמשים הרשומים למצלמה מסוימת כאשר משתמש אחר מתריע דרך ממשק הלקוח. מדובר בפעולה קריטית בהיבט של אבטחה וזיהוי אירועים בזמן אמת, ולכן יש לוודא שהפתרון יתבצע בצורה מהירה, אמינה, קלה להרחבה בהמשך ופשוטה להבנה.

קיימות מספר דרכים למימוש שליחת הודעות בטלגרם:

1. שימוש בספרייה כמו python-telegram-bot – מספק ממשק גבוה לשליחת הודעות, ניהול מצבים, קבלת הודעות ועוד. פתרון יעיל אך דורש ניהול אסינכרוני מסובך.

2. שימוש ב-telethon או pyrogram – ספריות מתקדמות המבוססות על התחברות ישירה ל-Telegram API. מעניקות שליטה רחבה אך מצריכות טיפול מורכב יותר ב-sessions ואימות.
3. שימוש פשוט ב-HTTP דרך הספרייה requests – שליחת קריאות POST רגילות ל-Telegram Bot API. פתרון פשוט וקל ליישום, במיוחד כאשר מדובר רק בשליחת הודעות טקסט.

הפנייה למקורות רלוונטים

- מידע על API של טלגרם:
<https://core.telegram.org/bots/api>
- דוקיומנטציית python-telegram-bot
<https://docs.python-telegram-bot.org/en/stable>
- מידע על telethon
<https://docs.telethon.dev/en/stable>
- מידע על pyrogram
<https://docs.pyrogram.org>

סקירת הפתרון הנבחר

הפתרון שנבחר הוא מימוש שליחת ההתראות באמצעות בקשת HTTP פשוטה דרך הספרייה requests לפונקציית sendMessage של Telegram Bot API.

האלגוריתם עובד כך:

- לכל משתמש במערכת יש chat_id שנשמר במסד הנתונים.
- בעת שליחת אזהרה, השרת עובר על רשימת כל המשתמשים הרשומים למצלמה.
- עבור כל משתמש, מבוצעת קריאה לפונקציה אשר יוצרת בקשת POST ל-Telegram עם טקסט ההתראה.
- אם הבקשה נכשלת, נלכדת החריגה ולא נגרמת קריסה למערכת.

נימוק הבחירה

- שיטה קלה מאוד להבנה ולתחזוקה.
- אין צורך לנהל חיבורים מורכבים, sessions, או הבנה עמוקה של מבנה האירועים בטלגרם.
- ניתן בקלות לשלב שינויים בהודעה, להרחיב לפורמטים נוספים, או להתאים לשרתים שונים.
- השימוש בקריאות HTTP רגילות מאפשר בקרה פשוטה על תגובת השרת.

שלילת פתרונות אלטרנטיביים

- python-telegram-bot – ספרייה פופולרית, אך לצורכי הפרויקט (שליחה חד כיוונית של הודעות) מדובר בפתרון כבד מדי שדורש ניהול event loop וידע בממשק מורכב יותר.
- telethon / pyrogram – מעניקות שליטה רחבה, אך דורשות חיבור לחשבון אישי של המשתמש ולא מתאימות לשימוש דרך Bot Token סטנדרטי בפרויקטים קטנים.

תיאור סביבת הפיתוח

כלי הפיתוח הדרושים לפיתוח:

- **פיצ'ארם (PyCharm):** סביבת פיתוח מתקדמת ונוחה לכתיבה וניפוי שגיאות בקוד Python, עם תמיכה באוטומציה ובדיבוג.
- **שפת תכנות פייתון:** שפה מונחית עצמים, קריאה ונוחה לפיתוח מערכות תקשורת וממשקים גרפיים.
- **גיט וגיטהב (Git & GitHub):** גיט הוא מערכת לניהול גרסאות שמאפשרת לעקוב אחרי שינויים בקוד. גיטהב הוא אתר שמבוסס על Git ומשמש לשיתוף וניהול פרויקטים באינטרנט ובו השתמשי בשביל ניהול גרסאות הפרויקט.
- **טרלו (Trello):** טרלו הוא כלי ניהול משימות חזותי שמבוסס על לוחות, רשימות וכרטיסים. נעשה בו שימוש בשביל לתכנן ולעקוב אחרי התקדמות העבודה בצורה פשוטה וברורה.

הסביבה והכלים הנדרשים לבדיקות:

- **מערכת הפעלה:** Windows להפעלת השרת והלקוחות בצורה תקינה.
- **מצלמות רשת:** לבדיקת שידור וידאו והעלאת תמונות חיות מהמצלמה לשרת.
- **מחשב פיזי או מספר מחשבים:** לבדיקת תקשורת בין שרת, משתמשים ומצלמות בזמן אמת.
- **חיבור אינטרנט:** לבדיקת שליחת הודעות, שידור וידאו ותקשורת מאובטחת.
- **חשבון טלגרם ובוט:** נדרש עבור שליחת קוד אימות והודעות מערכת דרך Telegram.
- **DB Browser:** לצפייה ישירה בנתוני המשתמשים והמצלמות במסד הנתונים במהלך שלב הבדיקות.

תיאור פרוטוקול התקשורת

פרוטוקול TCP הוא פרוטוקול תקשורת מבוסס חיבור הפועל בשכבת התעבורה, ומשמש להעברת נתונים בצורה אמינה ומסודרת בין שני צדדים. TCP מקים חיבור באמצעות תהליך בן שלושה שלבים בשם Three-Way Handshake ומבטיח שכל חבילת מידע שנשלחת תגיע ליעדה, תתקבל בסדר הנכון וללא שגיאות, תוך שימוש במנגנונים של תיקון שגיאות ובקרה. מבנה חבילת TCP כולל כותרת בגודל של לפחות 20 בתים (Bytes), שכוללת בין השאר את מספרי הפורטים של המקור והיעד, מספר סידורי בגודל 4 בתים, מספר אישור בגודל 4 בתים, אורך הכותרת, דגלים (כמו SYN, ACK, FIN), גודל חלון, סכום ביקורת (Checksum) ועוד. לאחר הכותרת מופיע גוף ההודעה המכיל את המידע שנשלח בפועל. פרוטוקול זה שימש בפרויקט שלי כבסיס להעברת נתונים בין שרת הפייתון לבין לקוחות המצלמות והמשתמשים, כדי להבטיח תקשורת מדויקת, מאובטחת ובלי איבוד מידע.

להלן פירוט של סוגי ההודעות שהעברתי בתקשורת ה-TCP בפרויקט:

- רצף הודעות login:

הרצף מתחיל בכך שנשלחת הודעת "login" מהלקוח לשרת. לאחר שמתקבל אישור שהשרת שולח חזרה ללקוח, הלקוח שולח לשרת את פרטי ההתחברות לפי הפורמט הבא:

"username password" וחשוב לשים לב שלא יכולים להיות רווחים בשם חשבון ובסיסמה. לאחר הודעה זו השרת משיב ללקוח או בהודעת הצלחה בה מפורט סוג החשבון של הלקוח "account_type logged in successfully", או בהודעת שגיאה מתאימה שתאפשר ללקוח לנסות להתחבר שוב עד שלושה ניסיונות. הודעות שגיאה שיכולות להישלח הן "No" "Wrong password" "Too many login attempts" "such username".

● רצף הודעות register:

הרצף מתחיל בכך שנשלחת הודעת "register" מהלקוח לשרת. לאחר שמתקבל אישור שהשרת שולח חזרה ללקוח, הלקוח שולח לשרת את פרטי ההתחברות לפי הפורמט הבא:

"username password account_type" וחשוב לשים לב שלא יכולים להיות רווחים בשם חשבון ובסיסמה. לאחר הודעה זו השרת משיב ללקוח או בהודעת הצלחה במידה וזה חשבון מסוג מצלמה, אם זה חשבון מסוג משתמש אז מתקבלת הודעת "verification code needed" מהשרת ועל הלקוח להכניס קוד אימות מבוט הטלגרם של המערכת ולאחר מכן תתקבל הודעת ההצלחה. יכולות להתקבל גם הודעות שגיאה בתהליך זה שכוללות: "No such" "Attempted username is taken" "verification Code".

● רצף הודעות העברת וידאו מצלמה לשרת:

בכל תמונה שנשלחת מהמצלמה לשרת, מצורף header מספריית struct שמביא לשרת מידע חיוני אודות התמונה שנשלחה כגון אורך ההודעה בשביל שהשרת יוכל לקבל אותה בצורה מדויקת ויעילה בלי צורך להשתמש בטיימאאוטים כדי לדעת מתי הפסיק לקבל את התמונה. לאחר שהשרת מקבל את התמונה במלואה הוא שולח הודעת אישור ללקוח המצלמה והתהליך חוזר חלילה.

● רצף הודעות קבלת וידאו מצלמה מהשרת:

בכל תמונה שנשלחת מהשרת למשתמש, מצורף header מספריית struct שמביא לשרת מידע חיוני אודות התמונה שנשלחה כגון אורך ההודעה בשביל שהמשתמש יוכל לקבל אותה בצורה מדויקת ויעילה בלי צורך להשתמש בטיימאאוטים כדי לדעת מתי הפסיק לקבל את התמונה. לאחר שהמשתמש מקבל את התמונה במלואה הוא שולח הודעת אישור לשרת שהוא מוכן לקבל עוד תמונה "ready_for_pic" והתהליך חוזר חלילה.

● רצף הודעות בקשת החלפת מצלמה:

לקוח מסוג משתמש שולח לשרת "switch_camera_request" והשרת שולח לו בתגובה "no_cameras" אם אין מצלמות זמינות במערכת, או הודעה בה מוצגים פרטים אודות המצלמה ממנה יקבל הלקוח את הוידאו כעת לפי הפורמט הבא:

"camera_name camera_id user_subscribed_to_camera". בשם המצלמה לא יכול להיות רווחים לכן אפשר לשלוח את ההודעה כך בצורה נוחה.

● הודעת הרשמה/ביטול הרשמה למצלמה:

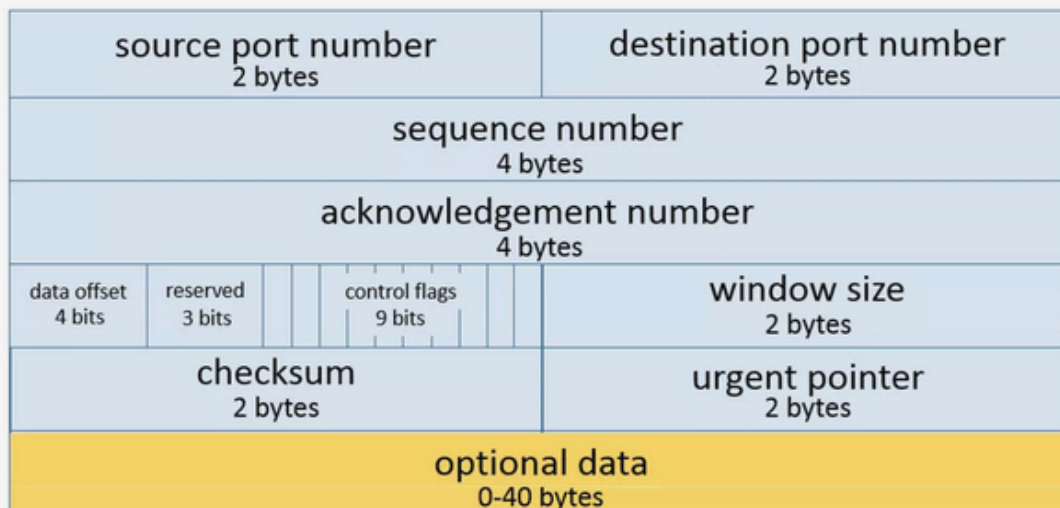
לקוח מסוג משתמש שולח לשרת הודעת "subscribe" פשוטה.

● הודעת התרעה לרשומים למצלמה:

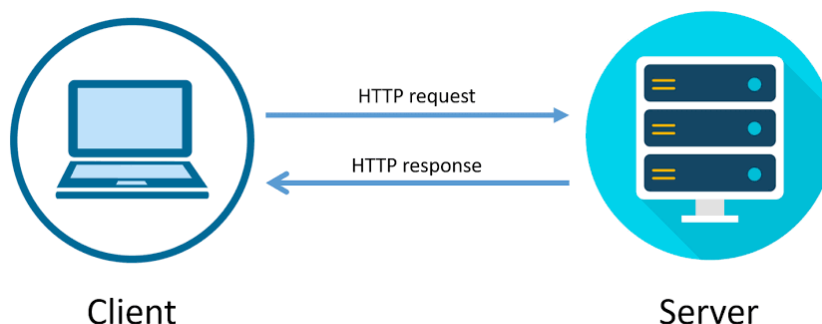
לקוח מסוג משתמש שולח לשרת הודעת "alert" פשוטה.

Transmission Control Protocol (TCP) Header

20-60 bytes

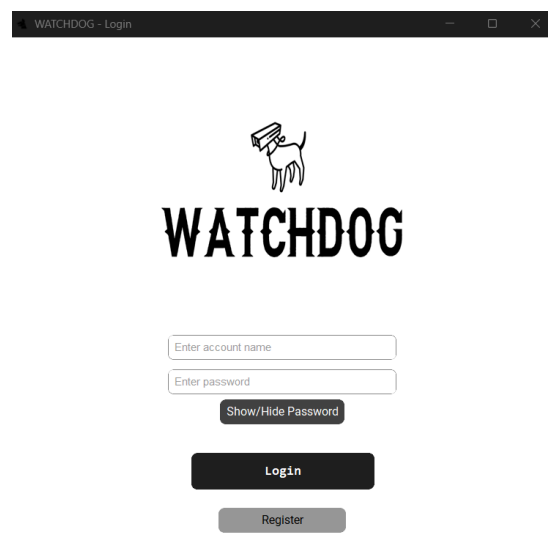


בקשות HTTP הן הדרך שבה לקוח מתקשר עם שרת אינטרנט. בקשה מורכבת מכתובת URL, שיטת בקשה, כותרות (Headers) ונתונים (Data או JSON). כאשר נשלחת בקשה לכתובת של API, השרת מקבל אותה, מבצע פעולה כלשהי (כמו שליחת הודעה), ומחזיר תשובה בפורמט JSON עם פרטי התוצאה. בפרויקט שלי השתמשתי בספריית requests של פייתון כדי לבצע קריאות ל-API של טלגרם, שליחת בקשת POST לכתובת `https://api.telegram.org/bot<BOT TOKEN HERE>/sendMessage` עם פרמטרים כגון מזהה הצ'אט (`chat_id`) והטקסט. בצורה זו, המערכת יודעת לשלוח הודעות בזמן אמת דרך טלגרם, באופן פשוט ומהיר.



תיאור מסכי המערכת

- מסך התחברות - במסך זה אפשר להתחבר לחשבון באמצעות שם חשבון וסיסמה. יש במסך כפתור להראות ולהסתיר את הסיסמה, כפתור התחברות למערכת וכפתור הרשמה למערכת שמעביר את הלוחץ למסך ההרשמה.



WATCHDOG - Login

WATCHDOG

Enter account name

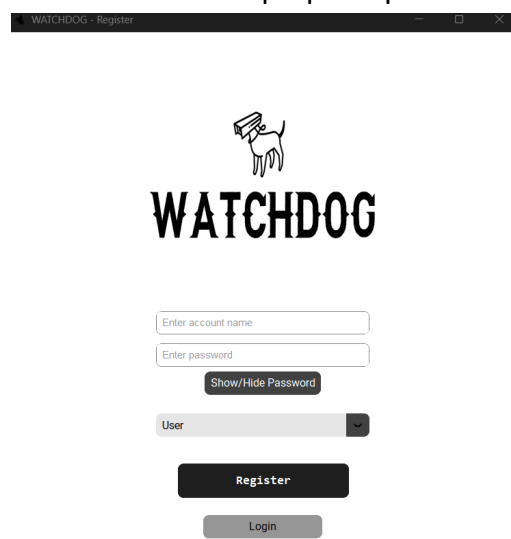
Enter password

Show/Hide Password

Login

Register

- מסך הרשמה - במסך זה אפשר ליצור חשבון חדש באמצעות שם, סיסמה ובחירת סוג חשבון (משתמש/מצלמה). יש במסך כפתור שמטרתו היא להראות ולהסתיר את הסיסמה, כפתור הרשמה למערכת וכפתור התחברות למערכת שמעביר למסך ההתחברות. לאחר יצירת חשבון משתמש קופץ חלון שמבקש קוד אימות מהטלגרם.



WATCHDOG - Register

WATCHDOG

Enter account name

Enter password

Show/Hide Password

User

Register

Login

- מסך משתמש - מסך זה מאפשר למשתמש המחובר למערכת לצפות במצלמות שונות המחוברות למערכת ולעשות כל מיני פעולות שונות. יש כפתור שאיתו אפשר להחליף מצלמה בין המצלמות השונות המחוברות במערכת, כפתור ששומר את התמונה הנוכחית שמוצגת במצלמה למחשב של המשתמש, כפתור שמאפשר למשתמש להירשם למצלמה ולקבל התרעות בנוגע אליה, כפתור שמאפשר למשתמש להתריע את כל הרשומים לאותה מצלמה שעליהם לשים לב למה שמתרחש בה/קיימת סכנה, וכפתור שמאפשר למשתמש לשקף את הוידאו של המצלמה בה הוא צופה.

WATCHDOG - User

No camera - try to switch cameras



No Video Available

Screenshot

Switch camera

Mirror footage

Subscribe to camera

Alert all camera subscribers

- מסך בחירת מצלמה - מסך זה מאפשר למפעיל חשבון מצלמה לבחון את המצלמות השונות שזמינות במחשב שלו לפני שמתחיל לשדר את הוידאו למערכת. אפשר במסך זה לבחור בתפריט בין המצלמות הזמינות ולבחון אותן באמצעות לחיצה על כפתור. לאחר שמפעיל חשבון המצלמה בוחר במצלמה הוא לוחץ על כפתור הבחירה במצלמה זו ועובר למסך המצלמה.

WATCHDOG - Camera Selection

Camera 0

Test Selected Camera

Select This Camera

- מסך מצלמה - מסך זה מאפשר למפעיל חשבון מצלמה להסתיר את הוידאו מהזווית שלו בשביל לחסוך משאבים במחשב שלו או להפעיל את הצפייה בוידאו מהמחשב שלו כדי שיוכל לראות מה הוא משדר לכל המשתמשים שצופים במצלמה זו. גם כן, יש כפתור שמאפשר לשקף את וידאו המצלמה שמשודר לשרת.



Camera hidden for you

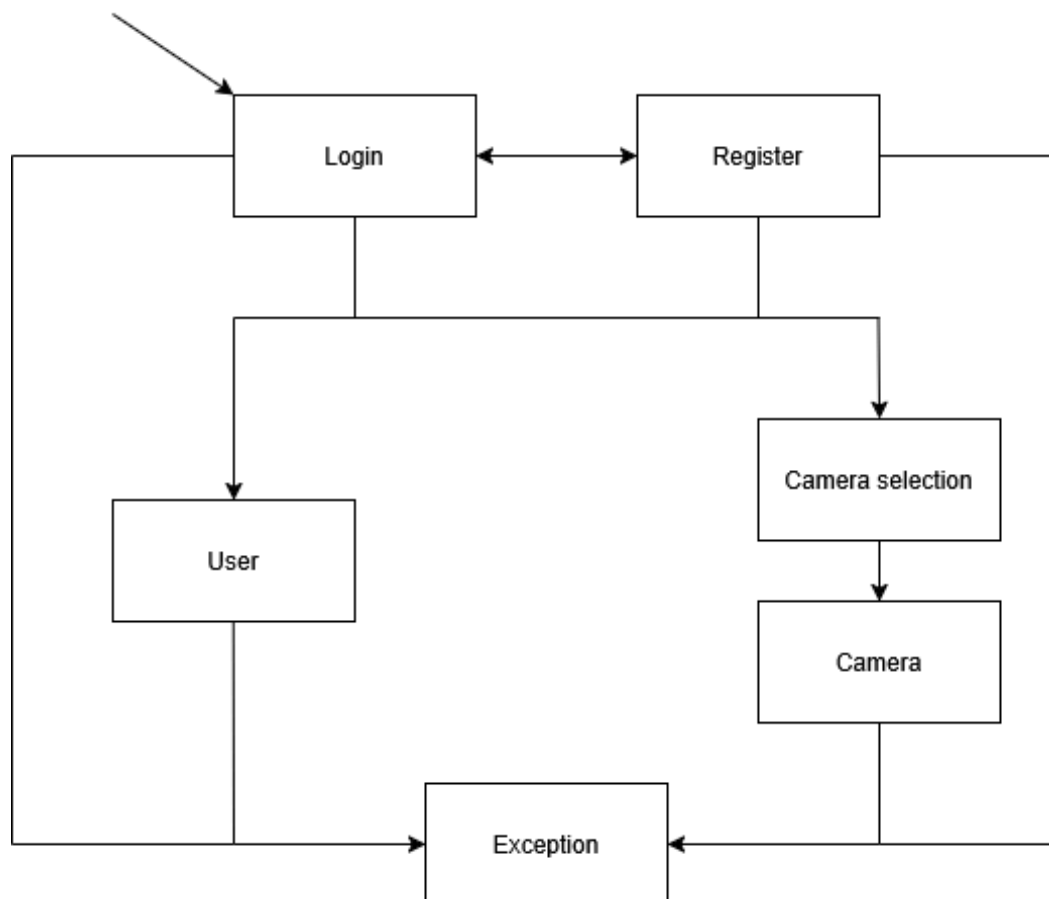
Show Camera Footage

Mirror Footage

- מסך שגיאה - מגיעים למסך זה כאשר יש שגיאה בתוכנה שמאלצת לסגור אותה, כמו: יותר מדי ניסיונות כושלים להתחברות למערכת שמובילים לחסימה מהמערכת או איבוד חיבור עם השרת. במסך זה רק ניתן לסגור את התוכנה.



Lost connection to server - Proceed by closing the app



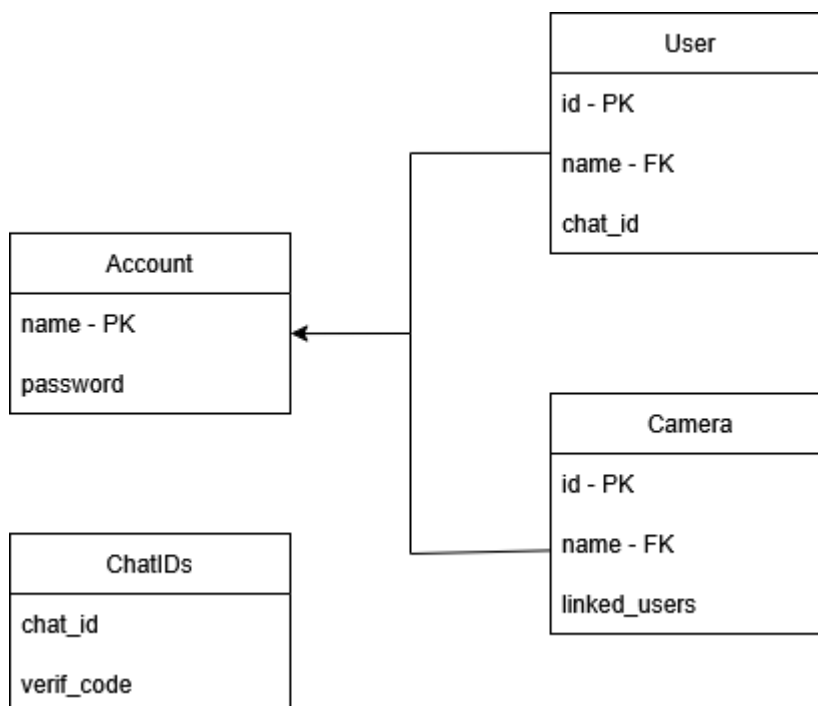
תיאור מבני הנתונים

בפרויקט זה קיימים מספר מבני נתונים בצד השרת. ביניהם נכללים: אובייקטים לניהול הלקוחות לפי סוג הלקוח (משתמש/מצלמה), רשימת הסוקטים של כל הלקוחות המחוברים לשרת, רשימת הסוקטים של כל המשתמשים המחוברים לשרת, רשימת הסוקטים של כל המצלמות המחוברות לשרת, סט של לקוחות שכתובת האיפי שלהם חסומה מהשרת (מסיבות אבטחה כמו חשד לניסיונות פריצה לחשבון) ובסיס נתונים.

שם מסד הנתונים בפרויקט זה הוא watchdog.db
טבלאות בסיס הנתונים, שדותיהם ודוגמאות לערכים בשדות:

טבלה	שדות	דוגמאות לערכים
ChatIDs	chat_id TEXT NOT NULL verif_code TEXT NOT NULL	1234567890 953264
Account	name TEXT NOT NULL UNIQUE PRIMARY KEY password TEXT NOT NULL	Israel ?Mdt=1pe][2t121a
User	id INTEGER AUTOINCREMENT PRIMARY KEY name TEXT NOT NULL FOREIGN KEY(Account) chat_id TEXT NOT NULL	1 Moshe 0123456789

5 LivingRoomCam Israel, Shlomo	id INTEGER AUTOINCREMENT PRIMARY KEY name TEXT NOT NULL FOREIGN KEY(Account) linked_users TEXT NOT NULL	Camera
--------------------------------------	---	--------



סקירת חולשות ואיומים

שכבת האפליקציה:

SQL Injection (הזרקת SQL):

המערכת משתמשת בבסיס נתונים מסוג sqlite3 לאחסון מידע על משתמשים, מצלמות והרשאות. כל השאילתות נכתבות תוך שימוש בפרמטרים מאובטחים, דבר המונע אפשרות להזרקת קוד.

Man In The Middle – מתקפת אדם בתווך:

כל ההודעות המועברות בין לקוחות לשרת מוצפנות בשיטת RSA. תמונות וידאו מוצפנות באמצעות AES, כאשר מפתח ההצפנה מועבר בצורה מאובטחת באמצעות RSA. בכך מובטחת סודיות ושלמות הנתונים המועברים גם במקרה של יירוט תקשורת.

DOS/DDOS – מתקפות מניעת שירות:

קיימת מניעה ראשונית של ניסיונות תקיפה על ידי חסימת כתובת ה-IP של לקוח לאחר שלושה ניסיונות התחברות כושלים. פתרון זה מפחית את הסיכון להצפת שרת על ידי התחברויות כושלות חוזרות. כמו כן, קיימת בשרת מגבלה של 10 לקוחות מחוברים בו זמנית בשביל למנוע עומס על השרת.

Hash הוא תהליך שבו ממירים מידע, כמו סיסמה, לערך קבוע באורך מסוים באמצעות אלגוריתם חד-כיווני. אלגוריתם SHA-256 הוא אחד מהאלגוריתמים הנפוצים ביותר, והוא מייצר גיבוב באורך

של 256 סיביות (64 תווים הקסדצימליים), ללא תלות באורך הקלט. תכונה חשובה של SHA-256 היא שכל שינוי קטן בקלט יוביל לתוצאה שונה לחלוטין, מה שמונע ניחוש של הסיסמה המקורית מתוך ערך הגיבוב. שמירת סיסמאות בבסיס הנתונים כערכי גיבוב מאפשרת להגן על המידע במקרה של דליפה או פריצה. גם אם תוקף יצליח להשיג גישה לבסיס הנתונים, הוא לא יוכל לראות את הסיסמאות עצמן אלא רק את ערכי הגיבוב, שכמעט בלתי אפשרי לפענח. לכן, שימוש ב-SHA-256 לשמירת סיסמאות הוא פתרון בטוח יותר מאשר שמירת סיסמאות בפורמט המקורי שלהן או בהצפנת RSA. משום כך נעשה שימוש בשכבת האפליקציה בשמירת הסיסמאות עם גיבוב בבסיס הנתונים.

שכבת התעבורה:

• שימוש בפרוטוקול TCP:

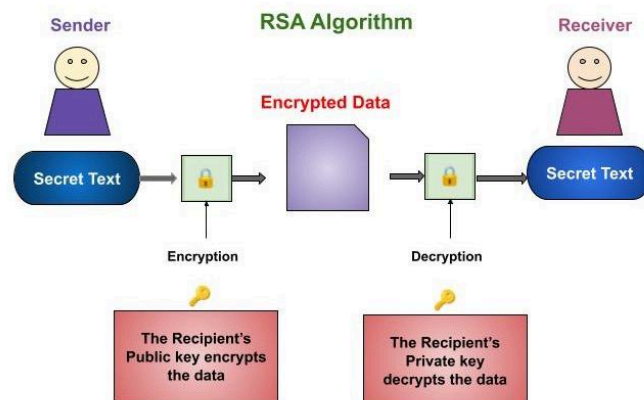
המערכת משתמשת בפרוטוקול TCP ליצירת חיבורים אמינים בין השרת ללקוחות. תהליך לחיצת היד המשולשת (Three-way handshake) של TCP מבטיח חיבור יציב ומאומת לפני העברת מידע.

• הצפנה בשכבת התעבורה:

כל המידע הנשלח בתקשורת בין השרת ללקוחות מוצפן באמצעות RSA. במקרה של תמונות, ההצפנה מתבצעת באמצעות AES, כאשר מפתח AES מועבר ב-RSA. כך מתקבלת גם מהירות הצפנה וגם אבטחה גבוהה.

RSA: שיטת הצפנה א-סימטרית, כלומר קיימים שני מפתחות נפרדים: מפתח ציבורי להצפנה, ומפתח פרטי לפענוח. זוהי טכניקת הצפנה נפוצה מאוד המשמשת בעיקר לשליחת מידע רגיש בצורה מאובטחת בערוצים לא בטוחים, ללא צורך בהעברת סיסמה משותפת מראש.

כאשר צד אחד (למשל לקוח) רוצה לשלוח מידע לשרת, הוא מצפין את המידע באמצעות המפתח הציבורי של השרת. רק השרת, שמחזיק את המפתח הפרטי, יכול לפענח את המידע. גם אם צד שלישי יירט את ההודעה, הוא לא יוכל להבין את תוכנה, מכיוון שאין לו את המפתח הפרטי.



RSA מתבססת על עיקרון מתמטי מתורת המספרים: קל להכפיל שני מספרים ראשוניים גדולים, אבל קשה מאוד לפרק את המכפלה שלהם בחזרה לגורמים. זהו הבסיס לקושי של פענוח ללא המפתח המתאים.

ההצפנה מתבצעת לפי הנוסחה: $c = m^{**e} \% n$

כאשר:

- c הוא המסר המוצפן
- m הוא המסר המקורי
- e הוא האקספוננט הציבורי (נכלל במפתח הציבורי)
- n הוא החלק השני של המפתח, הוא אותו דבר במפתח הציבורי והפרטי

הפענוח מתבצע לפי: $m = c^{**d} \% n$

כאשר:

- m הוא המסר המקורי
- c הוא המסר המוצפן
- d הוא האקספוננט הפרטי (המפתח הפרטי)
- n הוא החלק השני של המפתח, הוא אותו דבר במפתח הציבורי והפרטי

AES היא שיטת הצפנה סימטרית (Advanced Encryption Standard). גרסה AES-128 מצפינה כל בלוק מידע בגודל של 128 סיביות (16 בתים) באמצעות מפתח באורך 128 ביטים. בפרויקט נעשה שימוש בגרסת ECB, שבה כל בלוק מוצפן בנפרד (Electronic Code Book).

AES פועלת על נתונים המחולקים לבלוקים. כל בלוק עובר סדרה של סיבובים שבכל אחד מהם המידע משתנה בעזרת פעולות מתמטיות מורכבות ולא לינאריות. לאחר 10 סיבובים, הבלוק הופך לבלתי ניתן לזיהוי ולפענוח בלי המפתח.

שלבי ההצפנה:

1. פיצול לטקסט לבלוקים

הטקסט (Plaintext) מחולק לבלוקים של 16 בתים. אם הטקסט לא מתחלק בדיוק, מוסיפים Padding בשיטת PKCS#7 כדי להשלים. לפי שיטה זו, מוסיפים לסוף הבלוק האחרון את בית שערכו הוא מספר הבתים החסרים בשביל להשלים ל-16 כמספר הבתים.

2. המרת בלוק למטריצה

כל בלוק (16 בתים) הופך למטריצה של 4×4 בתים, המשמשת לעיבוד פנימי באלגוריתם.

p_0	p_4	p_8	p_{12}
p_1	p_5	p_9	p_{13}
p_2	p_6	p_{10}	p_{14}
p_3	p_7	p_{11}	p_{15}

k_0	k_4	k_8	k_{12}
k_1	k_5	k_9	k_{13}
k_2	k_6	k_{10}	k_{14}
k_3	k_7	k_{11}	k_{15}

3. הוספת מפתח ראשוני (Add round key)
מבצעים XOR בין הבלוק לבין המפתח הראשוני (הסודי).

x	y	x XOR y
0	0	0
0	1	1
1	0	1
1	1	0

4. 10 סבבים של הצפנה
בכל סבב מתבצעות הפעולות הבאות:

○ **Subtract bytes**: כל בת מוחלף לפי טבלת S-Box.

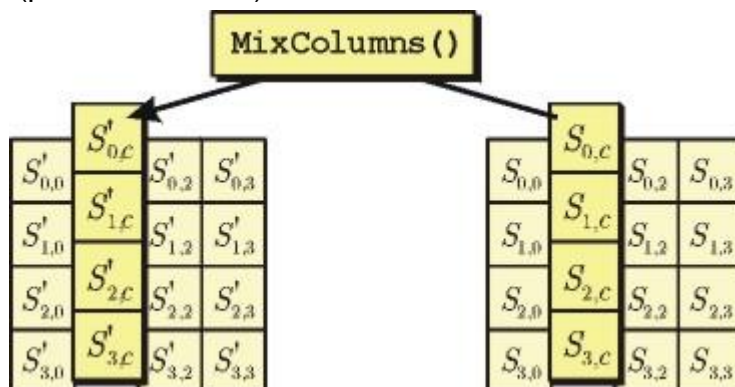
	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f
00	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
10	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
20	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
30	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
40	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
50	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
60	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
70	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
80	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
90	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a0	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b0	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c0	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d0	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e0	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f0	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

○ **Shift rows**: הזזה של שורות במטריצה כדי לפזר מידע.

S_0	S_4	S_8	S_{12}	← circular left shift with 0 step
S_1	S_5	S_9	S_{13}	← circular left shift with 1 steps
S_2	S_6	S_{10}	S_{14}	← circular left shift with 2 steps
S_3	S_7	S_{11}	S_{15}	← circular left shift with 3 steps

S_0	S_4	S_8	S_{12}	→	S_0	S_4	S_8	S_{12}
S_1	S_5	S_9	S_{13}		S_5	S_9	S_{13}	S_1
S_2	S_6	S_{10}	S_{14}		S_{10}	S_{14}	S_2	S_6
S_3	S_7	S_{11}	S_{15}		S_3	S_7	S_{11}	S_{15}

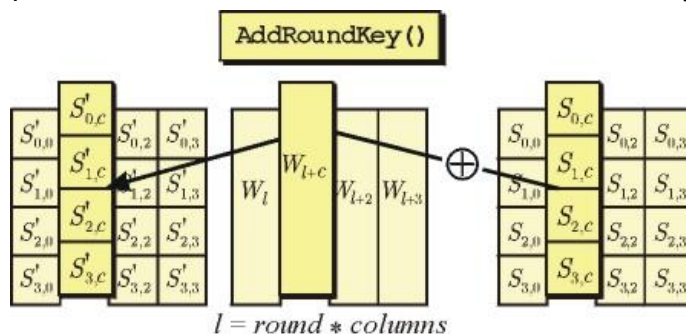
- **Mix columns:** שילוב מתמטי של עמודות (למעט בסבב האחרון).



$$b(x) = c(x) \otimes a(x)$$

$$\begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix}$$

- **Add round key:** הוספת מפתח חדש שנגזר מהמפתח המקורי.



5. איחוד הבלוקים

כל בלוק מוצפן בנפרד ומחברים את כולם כדי לקבל את הטקסט המוצפן (Ciphertext).

שלבי הפענוח:

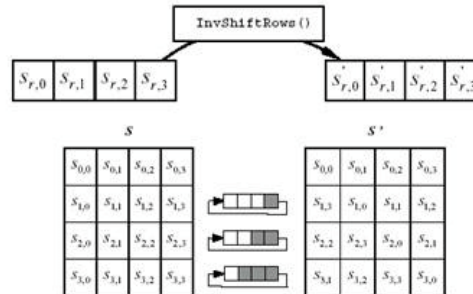
1. פיצול הטקסט המוצפן לבלוקים מחלקים את הטקסט המוצפן לבלוקים של 16 בתים.

2. המרת כל בלוק למטריצה בדיוק כמו בהצפנה.

3. 10 סבבי פענוח בסדר הפוך

הפענוח מתבצע על כל בלוק באמצעות פעולות הפוכות:

- **Add round key**: הסרת מפתח הסבב האחרון.
- **Inverse shift rows**: הזזת שורות חזרה למקור.



- **Inverse Subtract bytes**: שימוש ב-S-Box הפוכה.

	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f
00	52	09	6a	d5	30	36	a5	38	bf	40	a3	9e	81	f3	d7	fb
10	7c	e3	39	82	9b	2f	ff	87	34	8e	43	44	c4	de	e9	cb
20	54	7b	94	32	a6	c2	23	3d	ee	4c	95	0b	42	fa	c3	4e
30	08	2e	a1	66	28	d9	24	b2	76	5b	a2	49	6d	8b	d1	25
40	72	f8	f6	64	86	68	98	16	d4	a4	5c	cc	5d	65	b6	92
50	6c	70	48	50	fd	ed	b9	da	5e	15	46	57	a7	8d	9d	84
60	90	d8	ab	00	8c	bc	d3	0a	f7	e4	58	05	b8	b3	45	06
70	d0	2c	1e	8f	ca	3f	0f	02	c1	af	bd	03	01	13	8a	6b
80	3a	91	11	41	4f	67	dc	ea	97	f2	cf	ce	f0	b4	e6	73
90	96	ac	74	22	e7	ad	35	85	e2	f9	37	e8	1c	75	df	6e
a0	47	f1	1a	71	1d	29	c5	89	6f	b7	62	0e	aa	18	be	1b
b0	fc	56	3e	4b	c6	d2	79	20	9a	db	c0	fe	78	cd	5a	f4
c0	1f	dd	a8	33	88	07	c7	31	b1	12	10	59	27	80	ec	5f
d0	60	51	7f	a9	19	b5	4a	0d	2d	e5	7a	9f	93	c9	9c	ef
e0	a0	e0	3b	4d	ae	2a	f5	b0	c8	eb	bb	3c	83	53	99	61
f0	17	2b	04	7e	ba	77	d6	26	e1	69	14	63	55	21	0c	7d

- **Inverse mix columns**: שילוב עמודות הפוך (למעט בסבב הראשון).

A	0E	0B	0D	09
B	09	0E	0B	0D
C	0D	09	0E	0B
D	0B	0D	09	0E

=

A'
B'
C'
D'

4. הסרת Padding (אם קיים)

לאחר שחיברנו את כל הבלוקים המפוענחים – מסירים את הריפוד (במידה וקיים) כדי לקבל את הטקסט המקורי.

הפעלת המערכת:

- חולשות כלליות והתמודדות איתן:

- **הרצת קוד זדוני:** אין אפשרות להזרקת קוד דרך ממשק המשתמש – הקלטים מבוקרים ומוגבלים.
- **SQL Injection:** השימוש בפרמטרים בשאילתות מונע קלט לא מאובטח.
- **שמירת סיסמאות:** הסיסמאות אינן נשמרות בפורמט גלוי, אלא רק כערך hash בבסיס הנתונים.

מימוש הפרויקט

חלק א'

מודולים / מחלקות מיובאים:

- **cryptography** – הצפנת התמונות הנלכדות במצלמה באמצעות הצפנת AES.
- **customtkinter** – יצירת ממשק משתמש גרפי מותאם בצד הלקוח.
- **functools** – פונקציית reduce הנכללת במודול זה בשביל להפוך רשימה למחרוזת.
- **hashlib** – הצפנת הסיסמאות בבסיס הנתונים.
- **numpy** – עיבוד תמונה ופריימים מהוידאו.
- **opencv-python** – לכידת וידאו ממצלמת רשת, מניפולציית התמונה הנלכדת.
- **os** – גישה למערכת הקבצים, בשביל לשמור תמונות מסך.
- **pillow** – שמירת תמונות שנלכדו מהוידאו בפורמט גרפי.
- **pywin32** – הצגת Message Boxes.
- **python-telegram-bot** – שליחת הודעת ההתחלה בשיחת טלגרם בין משתמש לבוט.
- **random** – יצירת קוד אימות אקראי בשביל אימות משתמש דרך טלגרם.
- **requests** – שליחה/קבלה של נתונים בפרוטוקול HTTP בשביל שליחת התרעות בטלגרם למשתמשים שרשומים למצלמה.
- **socket** – יצירת חיבור TCP בין השרת ללקוחות.
- **sqlite3** – ניהול בסיס נתונים מקומי לאחסון משתמשים והרשאות.
- **struct** – אריזת נתונים בפורמט בינארי להעברה בתקשורת בשביל העברת תמונות.
- **threading** – ניהול תהליכים במקביל לאורך כל המערכת.
- **time** – פונקציית sleep בשביל להמתין כמות זמן מסוימת לפני שיתרחש דבר אחר.

מחלקות/מודולים שפיתחתי:

- מודול מסד נתונים - db.py

תפקיד: יצירת בסיס הנתונים בו נעשה שימוש בפרויקט וניהולו.

פעולות:

שם פעולה	טענת כניסה	טענת יציאה
main	אין	הפעולה יוצרת את מסד הנתונים בו נעשה שימוש בפרויקט
create_new_account	שם חשבון וסיסמה	במידה ולא קיים חשבון עם שם זה הפעולה יוצרת חשבון חדש בטבלת Account במסד הנתונים עם פרטים מוצפנים ואילו קיים אז מעלה שגיאה
create_new_user	שם משתמש ומזהה צ'אט טלגרם	הפעולה יוצרת משתמש חדש בטבלת User עם פרטים מוצפנים במסד נתונים
create_new_camera	שם מצלמה	הפעולה יוצרת מצלמה חדש בטבלת Camera עם פרטים מוצפנים במסד נתונים
create_new_chat_id_verif_code	מזהה צ'אט טלגרם וקוד אימות	הפעולה מוסיפה זוג מזהה צ'אט טלגרם וקוד אימות מוצפן לטבלת ChatIDs בבסיס הנתונים
login	שם חשבון וסיסמה	הפעולה בודקת אם קיים חשבון בבסיס הנתונים שעונה לפרטים שהתקבלו ומעלה שגיאות מתאימות אם לא
get_chat_id_from_verif_code	קוד אימות	הפעולה מחזירה את מזהה צ'אט הטלגרם שקשור לקוד האימות מטבלת ChatIDs ומוחק את השורה בה נמצאים
get_chat_id	שם משתמש	הפעולה מחזירה את מזהה צ'אט הטלגרם שקשור לשם המשתמש מטבלת User
get_user_id	שם משתמש	הפעולה מחזירה מזהה משתמש בהתאם לשם המשתמש מטבלת User
get_camera_id	שם מצלמה	הפעולה מחזירה מזהה מצלמה בהתאם לשם המצלמה מטבלת Camera
get_account_type	שם חשבון	הפעולה מחזירה את סוג החשבון בהתאם לשם החשבון (משתמש/מצלמה)
get_camera_subscriptions	מזהה מצלמה	הפעולה מחזירה רשימה של כל שמות המשתמשים שרשומים למצלמה עם מזהה המצלמה המתקבל
subscribe_to_camera	שם משתמש ומזהה מצלמה	הפעולה רושמת/מוחקת הרשמה של שם המשתמש מהמצלמה עם מזהה המצלמה שהתקבל
hash_password	סיסמה	הפעולה מצפינה באמצעות hash את הסיסמה שהתקבלה ומחזירה את הסיסמה המוצפנת.

- מודול הצפנה - encryption.py

תפקיד - הצפנה ופענוח של הודעות העוברות בין השרת ללקוח במהלך הפרויקט.

פעולות:

שם פעולה	טענת כניסה	טענת יציאה
rsa_encrypt	מחרוזת להצפנה ומפתח ציבורי	הפעולה מממשת הצפנת RSA למחרוזת שהתקבלה לפי המפתח הציבורי ומחזירה מחרוזת מוצפנת
rsa_decrypt	מחרוזת מוצפנת ומפתח פרטי	הפעולה מממשת פענוח RSA למחרוזת המוצפנת שהתקבלה לפי המפתח הפרטי ומחזירה מחרוזת מפוענחת
aes_ecb_encrypt	מידע להצפנה בביתים ומפתח סימטרי	הפעולה מצפינה את המידע שקיבלה באמצעות AES לפי המפתח שקיבלה ומחזירה את המידע המוצפן
aes_ecb_decrypt	מידע לפענוח בביתים ומפתח סימטרי	הפעולה מפענחת את המידע שקיבלה באמצעות AES לפי המפתח שקיבלה ומחזירה את המידע המפוענח

- בוט טלגרם - telegram_bot.py

תפקיד - טיפול באימות מול משתמשים באמצעות בוט טלגרם. שליחת הודעות וביצוע זיהוי ראשוני.

תכונה:

שם תכונה	תפקיד ושימוש
BOT_TOKEN	מחרוזת של טוקן הטלגרם של הבוט שאיתו אפשר להשתמש בבוט דרך הקוד בפיתוח לשליחת הודעות ואינטרקציה עם משתמשים

פעולות:

שם פעולה	טענת כניסה	טענת יציאה
send_telegram_message	מזהה צ'אט טלגרם והודעה לשלוח	הפעולה שולחת הודעה למשתמש דרך טלגרם, מחזירה אמת אם הצליחה ושקר אם לא
generate_six_digit_code	אין	מחזירה קוד אקראי בן 6 ספרות
start	אובייקטים של טלגרם	שולחת קוד אימות למשתמש שמתחיל שיחה עם הבוט בטלגרם ושומרת אותו במסד הנתונים בקישור למזהה צ'אט הטלגרם שאליו שלחה את קוד האימות
main	אין	הפעולה מאפשרת לבוט הטלגרם להאזין לתחילת שיחה עם משתמשים בטלגרם

- מחלקת לקוח - Client

תפקיד - ניהול תכונות לקוח בצד השרת.

תכונות:

שם תכונה	תפקיד ושימוש
client_socket	לאפשר גישה לסוקט של הלקוח בשביל לקיים תקשורת איתו דרך השרת
name	לאפשר גישה לשם החשבון של הלקוח לשימוש במהלך קוד השרת

- מחלקת משתמש - User

תפקיד - ניהול תכונות לקוח משתמש בצד השרת.

תכונות:

שם תכונה	תפקיד ושימוש
----------	--------------

client_socket	לאפשר גישה לסוקט של המשתמש בשביל לקיים תקשורת איתו דרך השרת
username	לאפשר גישה לשם החשבון של המשתמש לשימוש במהלך קוד השרת
user_id	לאפשר גישה למזהה המשתמש של המשתמש לשימוש במהלך קוד השרת
chat_id	לאפשר גישה למזהה צ'אט הטלגרם של המשתמש בשביל שליחת הודעות בטלגרם למשתמש
ready_for_pic	לאפשר לשרת לדעת מתי המשתמש מצפה לקבל תמונות מצלמה בתקשורת

- מחלקת מצלמה - Camera

תפקיד - ניהול תכונות לקוח מצלמה בצד השרת.

תכונות:

שם תכונה	תפקיד ושימוש
client_socket	לאפשר גישה לסוקט של המצלמה בשביל לקיים תקשורת איתה דרך השרת
camera_name	לאפשר גישה לשם החשבון של המצלמה לשימוש במהלך קוד השרת
camera_id	לאפשר גישה למזהה המצלמה של המצלמה לשימוש במהלך קוד השרת
encryption_key	לאפשר למשתמש לקבל את מפתח ה-AES שאיתו מצפינים את תמונות המצלמה כדי לפענח את התמונות

- מחלקת App

תפקיד - ניהול התוכנה והמסכים השונים בצד הלקוח.

תכונות:

שם תכונה	תפקיד ושימוש
login_frame	מסך התחברות
register_frame	מסך הרשמה
user_frame	מסך משתמש
camera_selection_frame	מסך בחירת מצלמה
camera_frame	מסך מצלמה
exception_frame	מסך שגיאה

פעולות:

שם פעולה	טענת כניסה	טענת יציאה
init	אין	הפעולה יוצרת את ממשק המשתמש ומעלה את מסך ההתחברות ללקוח
close_app	אין	הפעולה סוגרת את התקשורת עם השרת, את ממשק המשתמש וסוגרת את התוכנית
show_login_frame	אין	הפעולה מעלה את מסך ההתחברות
show_register_frame	אין	הפעולה מעלה את מסך ההרשמה
show_user_frame	אין	הפעולה מעלה את מסך המשתמש ומתחילה את פעולתו
show_camera_selection_frame	אין	הפעולה מעלה את מסך בחירת המצלמה ומתחילה את פעולתו
show_camera_frame	מספר המצלמה בה נבחר בתפריט בחירת המצלמות	הפעולה מעלה את מסך המצלמה ומתחילה את פעולתו
show_exception_frame	אין	הפעולה מעלה את מסך השגיאה ומתחילה את פעולתו

- מחלקת ExceptionFrame

תפקיד - חלון שגיאה בתוכנית בצד הלקוח.

תכונות:

שם תכונה	תפקיד ושימוש
app	האפליקציה הראשית בממשק למשתמש
logo_label	להראות את לוגו הפרויקט במסך השגיאה
error_msg	להראות את הודעת השגיאה במסך השגיאה

פעולות:

שם פעולה	טענת כניסה	טענת יציאה
init	האפליקציה הראשית	הפעולה יוצרת את מסך השגיאה
update_error_msg	שגיאה	הפעולה משנה את הודעת השגיאה שמוצגת במסך השגיאה

• מחלקת LoginForm

תפקיד - חלון התחברות בתוכנית בצד הלקוח.

תכונות:

שם תכונה	תפקיד ושימוש
app	האפליקציה הראשית בממשק למשתמש
logo_label	להראות את לוגו הפרויקט במסך ההתחברות
username_entry	מקום הכנסת שם החשבון בהתחברות
password_entry	מקום הכנסת הסיסמה בהתחברות
show_password_button	כפתור שמראה/מסתיר את הסיסמה (מסתיר עם כוכביות)
login_button	כפתור התחברות לחשבון
register_button	כפתור מעבר למסך ההרשמה

פעולות:

שם פעולה	טענת כניסה	טענת יציאה
init	האפליקציה הראשית	הפעולה יוצרת את מסך

ההתחברות		
הפעולה מתחילה תהליכון להתחברות לחשבון	אין	login_to_account_thread_start
הפעולה מתחברת לחשבון ומציגה הודעות שגיאה בהתאם ומעבירה למסכים השונים בהתאם לסוג החשבון	אין	login_to_account

• מחלקת RegisterFrame

תפקיד - חלון הרשמה בתוכנית בצד הלקוח.

תכונות:

שם תכונה	תפקיד ושימוש
app	האפליקציה הראשית בממשק למשתמש
logo_label	להראות את לוגו הפרויקט במסך ההרשמה
username_entry	מקום הכנסת שם החשבון בהרשמה
password_entry	מקום הכנסת הסיסמה בהרשמה
show_password_button	כפתור שמראה/מסתיר את הסיסמה (מסתיר עם כוכביות)
account_type	תפריט בחירת סוג חשבון (משתמש/מצלמה)
register_button	כפתור יצירת חשבון
login_button	כפתור מעבר למסך ההתחברות

פעולות:

שם פעולה	טענת כניסה	טענת יציאה
init	האפליקציה הראשית	הפעולה יוצרת את מסך ההתחברות
register_account_thread_start	אין	הפעולה מתחילה תהליכון הרשמה
register_account	אין	הפעולה יוצרת משתמש ומציגה הודעות שגיאה בהתאם ומעבירה למסכים השונים בהתאם לסוג החשבון או

לשגיאה		
--------	--	--

- מחלקת UserFrame

תפקיד - חלון משתמש בתוכנית בצד הלקוח.

תכונות:

שם תכונה	תפקיד ושימוש
app	האפליקציה הראשית בממשק למשתמש
camera_title	תיאור המצלמה הנוכחית
receiving_footage	האם המשתמש מקבל סרטון מצלמה מהשרת
receiving_footage_thread	תהליכון קבלת סרטון מצלמה מהשרת
image_to_display_in_numpy_array	התמונה שתוצג למשתמש במסך במערך נאמפיי
camera_encryption_key	מפתח ה-AES של המצלמה הנוכחית
camera_label	וידאו המצלמה המוצג למשתמש
screenshot_button	כפתור שמירת תמונה נוכחית
switch_camera_button	כפתור החלפת מצלמה
mirrored	האם המשתמש בחר שהסרטון יהיה משוקף
mirror_button	כפתור שיקוף תמונה מצד השמתמש
subscribe_button	כפתור הרשמה/ ביטול הרשמה למצלמה
subscribed	האם המשתמש רשום למצלמה הנוכחית
alert_button	כפתור התרעה לכל הרשומים למצלמה הנוכחית

פעולות:

שם פעולה	טענת כניסה	טענת יציאה
init	האפליקציה הראשית	הפעולה יוצרת את מסך המשתמש
start_receiving_footage	אין	הפעולה מנסה להחליף מצלמה ומראה את סרטון המצלמה הנוכחית

הפעולה משנה את התכונה שקובעת אם הסרטון ישוקף	אין	mirror_footage
הפעולה מתחילה את תהליכון שמירת התמונה	אין	screenshot_start_thread
הפעולה שומרת את התמונה הנוכחית במחשב המשתמש ופותחת אותה בחלון מעל מסך המשתמש	אין	screenshot
הפעולה מתחילה את תהליכון התרעת הרשומים למצלמה הנוכחית	אין	alert_start_thread
הפעולה שולחת בקשת התרעה לרשומי המצלמה הנוכחית לשרת	אין	alert_camera_subscribers
הפעולה מתחילה את תהליכון התרעת הרשמה/ביטול הרשמה למצלמה הנוכחית	אין	sub_start_thread
הפעולה שולחת בקשה לשרת להרשמה/ביטול הרשמה למצלמה הנוכחית בהתאם למצב ההרשמה הנוכחי של המשתמש	אין	subscribe_to_camera
הפעולה מתחילה את תהליכון החלפת המצלמה	אין	switch_camera_request_start_thread
הפעולה מפסיקה את קבלת הוידאו מהשרת ומנסה להחליף מצלמה בתקשורת עם השרת	אין	switch_camera_request
הפעולה מקבלת את סרטון המצלמה הנוכחית מהשרת	אין	receive_camera_footage
הפעולה אחראית על להראות את וידאו המצלמה שמתקבל מהשרת בממשק המשתמש	אין	show_camera_footage

● מחלקת CameraSelectionFrame

תפקיד - חלון בחירת מצלמה בתוכנית בצד הלקוח.

תכונות:

שם תכונה	תפקיד ושימוש
app	האפליקציה הראשית בממשק למשתמש
camera_active	האם יש מצלמה פועלת בחלון זה
video_capture	המצלמה שנבחרה בחלון זה
camera_index	תפריט בחירת מצלמה
test_camera_button	כפתור בדיקת המצלמה שנבחרה
camera_label	וידאו המצלמה המוצג
select_camera_button	כפתור בחירת מצלמה ומעבר לחלון המצלמה

פעולות:

שם פעולה	טענת כניסה	טענת יציאה
init	האפליקציה הראשית	הפעולה יוצרת את חלון בחירת המצלמה
update_camera_indexes	אין	הפעולה מעדכנת את תפריט בחירת המצלמות בהתאם למצלמות הזמינות במחשב
start_camera	אין	הפעולה מתחילה תהליכון לבדיקת המצלמה עם מספר המצלמה שנבחר
stop_camera	אין	הפעולה עוצרת את וידאו המצלמה הנבדקת
test_camera	מספר המצלמה לבדיקה	הפעולה מתחילה את בדיקת המצלמה עם המזהה שקיבלה
count_connected_cameras	אין	הפעולה מחזירה את מספר המצלמות הזמינות במחשב

- מחלקת CameraFrame

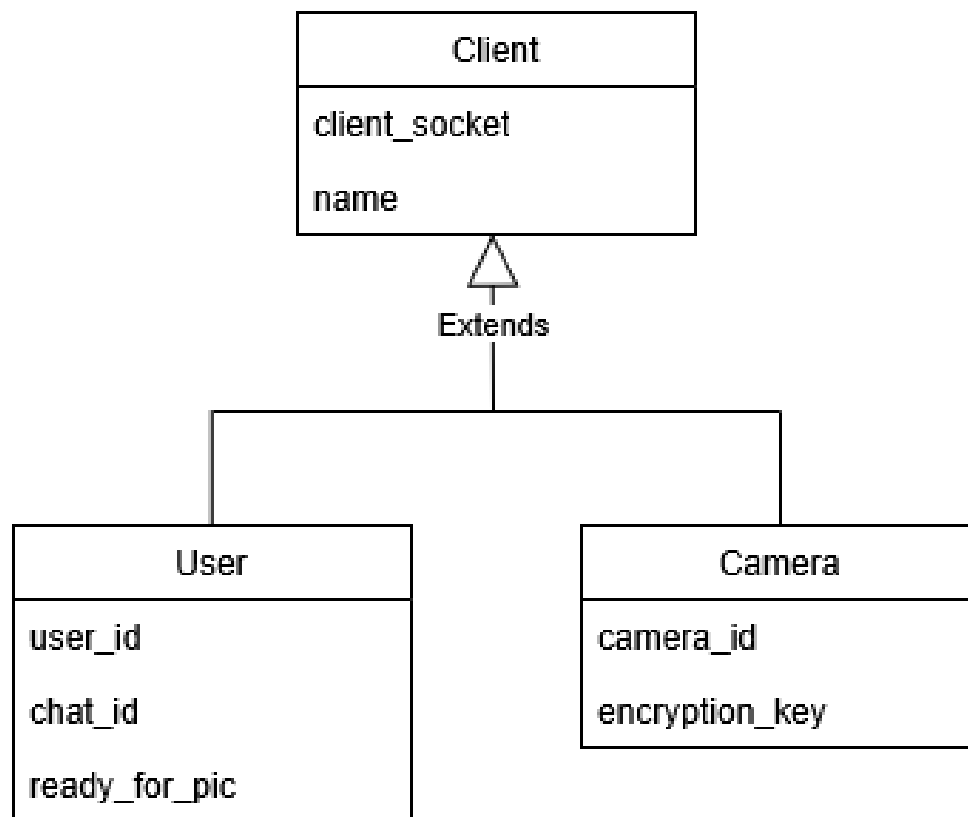
תפקיד - חלון מצלמה בתוכנית בצד הלקוח.

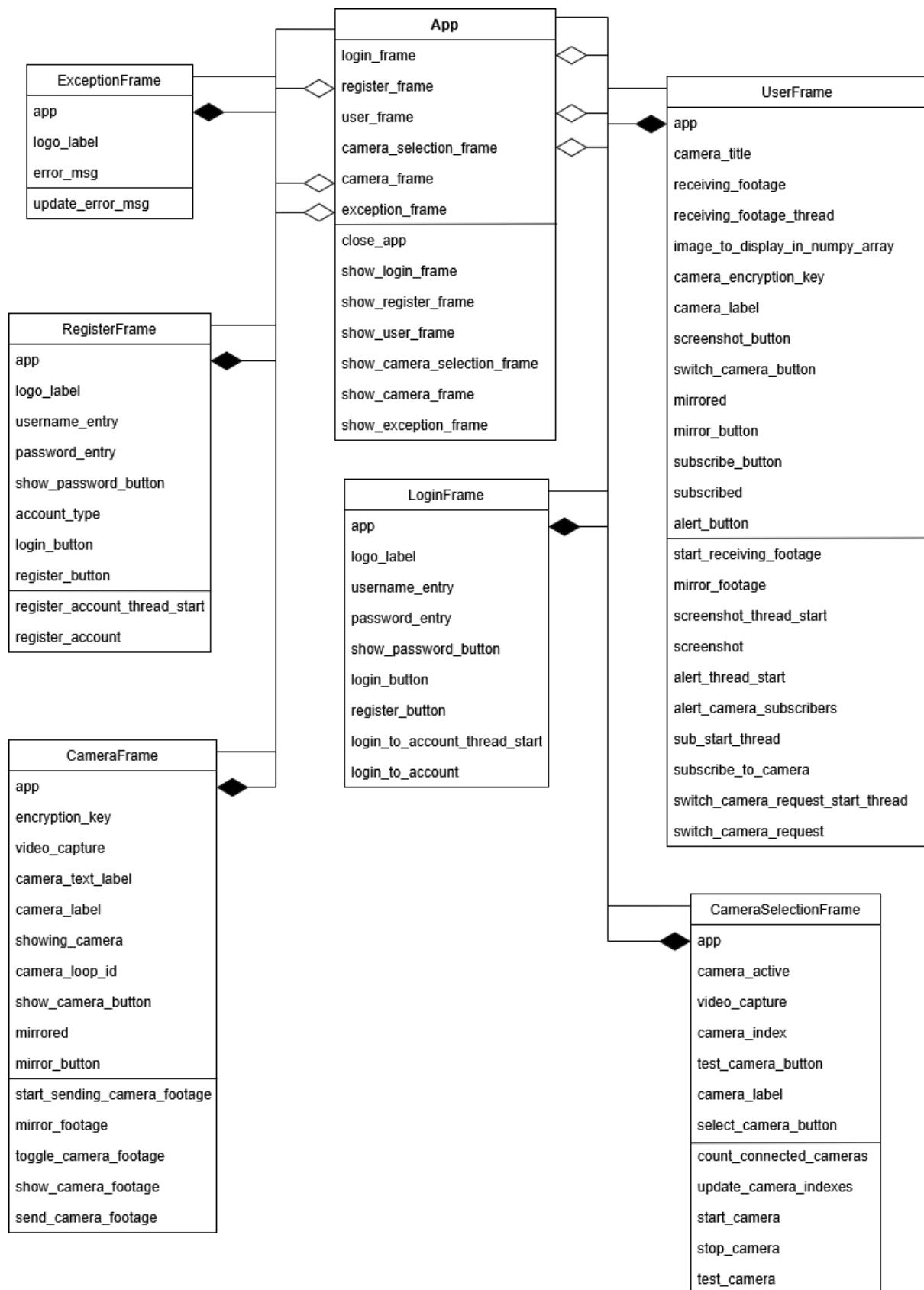
תכונות:

שם תכונה	תפקיד ושימוש
app	האפליקציה הראשית בממשק למשתמש
encryption_key	מפתח ה-AES להצפנה של וידאו המצלמה
video_capture	המצלמה שממנה נלקח הוידאו
camera_text_label	טקסט המצלמה מוסתרת
camera_label	וידאו המצלמה
showing_camera	האם המצלמה מוצגת
camera_loop_id	מזהה לולאת הצגת וידאו המצלמה
show_camera_button	כפתור להראות/להסתיר וידאו מצלמה
mirrored	האם המצלמה משוקפת
mirror_button	כפתור שיקוף וידאו מצלמה

פעולות:

שם פעולה	טענת כניסה	טענת יציאה
init	האפליקציה הראשית	הפעולה יוצרת את חלון בחירת המצלמה
start_sending_camera_footage	מזהה מצלמה	הפעולה מייצרת את מפתח ההצפנה ומתחילה לשלוח לשרת את וידאו המצלמה שנבחרה במסך הקודם
mirror_footage	אין	הפעולה משנה את התכונה שקובעת אם הסרטון ישוקף
toggle_camera_footage	אין	הפעולה מסתירה את וידאו המצלמה או קוראת לפעולת להראות וידאו מצלמה
show_camera_footage	אין	הפעולה אחראית על להראות את וידאו המצלמה בחלון
send_camera_footage	אין	הפעולה שולחת את וידאו המצלמה לשרת





חלק ב'

- שליחה וקבלה של וידאו המצלמה - יכולת מרכזית וחשובה מאוד בפרויקט, העברת וידאו המצלמה בתקשורת מלקוח מצלמה לשרת ולקבל את אותו וידאו חזרה מהשרת בתור חשבון משתמש.

```
def send_camera_footage(self):
    try:
        while True:
            ret, frame = self.video_capture.read()
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY) # Turn frame
black and white

            if self.mirrored:
                frame = numpy.flip(frame, axis=1)

            # Compress the frame using JPEG
            encode_param = [int(cv2.IMWRITE_JPEG_QUALITY), 30]
            _, encoded_frame = cv2.imencode('.jpg', frame, encode_param)

            frame_bytes = encoded_frame.tobytes()
            encrypted_frame_bytes = aes_ecb_encrypt(self.encryption_key,
frame_bytes)

            message_size = struct.pack("L", len(encrypted_frame_bytes))
            client_socket.sendall(message_size + encrypted_frame_bytes)

            # Image received confirmation -> send next one
            client_socket.recv(1024)
        except (BrokenPipeError, ConnectionResetError, socket.timeout,
OSError):
            self.app.show_exception_frame("Lost connection to server")
```

```
def receive_camera_footage(self):
    while self.receiving_footage:
        try:

client_socket.send(rsa_encrypt(constants.client_to_server_public_key,
"ready_for_pic").encode())

            received_data = b""
            payload_size = struct.calcsize("L")

            while len(received_data) < payload_size:
                received_data += client_socket.recv(4096)

            packed_msg_size = received_data[:payload_size]
```

```

received_data = received_data[payload_size:]
msg_size = struct.unpack("L", packed_msg_size)[0]

while len(received_data) < msg_size:
    received_data += client_socket.recv(4096)

frame_data = received_data[:msg_size]
decrypted_frame_data =
aes_ecb_decrypt(self.camera_encryption_key, frame_data)

nparr = numpy.frombuffer(decrypted_frame_data, numpy.uint8)
frame = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

if self.mirrored:
    frame = numpy.flip(frame, axis=1)

self.image_to_display_in_numpy_array = frame
except socket.timeout:
    MessageBox(0, "Connection to camera lost", "Camera
Unavailable", MB_ICONERROR)
    self.image_to_display_in_numpy_array =
numpy.array(Image.open("no_video.jpg"))
    self.camera_title.configure(text="No camera - try to switch
cameras")
    self.receiving_footage = False
except (BrokenPipeError, ConnectionResetError, OSError):
    self.app.show_exception_frame("Lost connection to server")

```

- שליחת התרעות בטלגרם - יכולת מרכזית וחשובה מאוד בפרויקט, שליחת ההודעות בטלגרם לכל הרשומים למצלמה מסוימת בה החליטו להתריע, על ידי שימוש ב-API של טלגרם.

```

def send_alert_messages(subs, alerter_name, camera_id):
    for sub in subs:
        if sub != alerter_name:
            chat_id = int(db.get_chat_id(sub))
            try:
                telegram_bot.send_telegram_message(chat_id, f"User
{alerter_name} has decided to alert all users of the camera with the id
of {camera_id} ")
            except Exception as e:
                pass

```

```

def send_telegram_message(chat_id, message):
    url = f"https://api.telegram.org/bot{BOT_TOKEN}/sendMessage"
    payload = {"chat_id": chat_id, "text": message} # json

```

```
try:
    response = requests.post(url, data=payload)
    response.raise_for_status()
    return True
except requests.RequestException as e:
    return False
```

- **בדיקת מצלמות פנויות** - קטע קוד שרלוונטי ללקוחות מסוג מצלמה, נבדקות המצלמות הזמינות לשימוש בתוכנה ולאחר מכן מוצגות בממשק המשתמש כדי לבחור מהן.

```
def count_connected_cameras(self):
    camera_count = 0
    looking_for_cameras = True
    cameras_indexes = []

    while looking_for_cameras:
        cap = cv2.VideoCapture(camera_count)
        if cap.isOpened():
            ret, frame = cap.read()
            if ret:
                cameras_indexes.append(camera_count)
                cap.release()
            else:
                cap.release()
                looking_for_cameras = False

        camera_count += 1

    return cameras_indexes
```

- **שמירת תמונה** - קטע קוד שרלוונטי ללקוחות מסוג משתמש, כאשר המשתמש רוצה לשמור תמונה במחשב שלו אז קטע הקוד הבא לוקח את התמונה הנוכחית שמוצגת למשתמש ונותנת שם מתאים לקובץ בהתאם לתמונות שכבר שמורות אצלו. גם כן, התמונה מוצגת לו בחלון מעל התוכנה שרצה.

```
def screenshot(self):
    image = Image.fromarray(self.image_to_display_in_numpy_array)

    screenshots_folder = os.getcwd()
    path = os.path.join(screenshots_folder, '*')
    file_name = None

    files = win32file.FindFilesW(path)
    for f in files:
        if ".jpg" in f[8] and "screenshot" in f[8]:
            file_name = f[8]
```

```
if file_name is None:
    file_name = "screenshot0.jpg"
else:
    file_name = "screenshot" +
str(int(file_name.split('.')[0].split('screenshot')[1]) + 1) + ".jpg"

image.save(file_name)

toplevel = CTkToplevel(self)
toplevel.attributes("-topmost", True)
toplevel.title(file_name)

photo_image = ImageTk.PhotoImage(image=image)
image_label = CTkLabel(toplevel, text="", image=photo_image)
image_label.pack()
```

חלק ג'

הבדיקות שתוכננו בשלב האפיון:

1. בדיקת התחברות משתמש תקין

- **מטרת הבדיקה:** לוודא שמשתמש קיים יכול להתחבר עם פרטים נכונים.
- **מה בוצע בפועל:** הוזנו שם משתמש וסיסמה תקינים לחלון ההתחברות.
- **תוצאת הבדיקה:** המשתמש נכנס בהצלחה למערכת.
- **בעיות ופתרון:** לא התגלו בעיות.

2. בדיקת התחברות עם סיסמה שגויה

- **מטרת הבדיקה:** לבדוק האם המערכת מזהה סיסמה שגויה ומונעת כניסה.
- **מה בוצע בפועל:** הוזנה סיסמה שגויה שלוש פעמים ברצף.
- **תוצאת הבדיקה:** המשתמש נחסם זמנית ולא התאפשרה כניסה.
- **בעיות ופתרון:** לא התגלו בעיות.

3. בדיקת הרשמה וקבלת קוד אימות

- **מטרת הבדיקה:** לבדוק אם נשלח קוד אימות לטלגרם לאחר הרשמה.
- **מה בוצע בפועל:** בוצעה הרשמה דרך הממשק, הקוד נשלח לבוט בטלגרם.
- **תוצאת הבדיקה:** ההודעה לא התקבלה.
- **בעיות ופתרון:** היה צריך להחליף לספרייה אחרת בשביל בוט הטלגרם שעובדים איתה עם קוד אסינכרוני מסובך יותר ממה שתוכנן מראש.

4. בדיקת התחברות מצלמה לשרת

- **מטרת הבדיקה:** לוודא שמצלמה יכולה להתחבר ולהעביר שידור.
- **מה בוצע בפועל:** הופעלה מצלמה והתחברה לשרת דרך לקוח מסוג מצלמה.
- **תוצאת הבדיקה:** הזרמת תמונה בוצעה בהצלחה.
- **בעיות ופתרון:** לא התגלו בעיות.

5. בדיקת צפייה בשידור חי

- **מטרת הבדיקה:** לוודא שהמשתמש רואה את שידור המצלמה.
- **מה בוצע בפועל:** משתמש התחבר וצפה בתמונה מהמצלמה בזמן אמת.
- **תוצאת הבדיקה:** התמונה הוצגה בזמן אמת על המסך.
- **בעיות ופתרון:** לא התגלו בעיות.

6. בדיקת הרשמה עם שם משתמש קיים

- **מטרת הבדיקה:** לוודא שלא ניתן להירשם עם שם שכבר קיים.
- **מה בוצע בפועל:** בוצעה הרשמה עם שם משתמש קיים במסד הנתונים.
- **תוצאת הבדיקה:** המערכת החזירה הודעת שגיאה.
- **בעיות ופתרון:** לא התגלו בעיות.

7. בדיקת הרשמה עם קוד אימות שגוי

- **מטרת הבדיקה:** לבדוק האם המערכת מזהה קוד אימות שגוי.
- **מה בוצע בפועל:** הוזן קוד שגוי בשלב האימות.
- **תוצאת הבדיקה:** הרשמה לא התבצעה, הודעת שגיאה הוצגה.
- **בעיות ופתרון:** לא התגלו בעיות.

8. בדיקת התנתקות הלקוח מהשרת

- **מטרת הבדיקה:** לוודא שהשרת מטפל נכון בהתנתקות יזומה.
- **מה בוצע בפועל:** בוצעה התנתקות דרך כפתור יציאה בממשק.
- **תוצאת הבדיקה:** השרת לא סגר את החיבור בצורה תקינה.
- **בעיות ופתרון:** בדיקה זו נערכה לאחר בדיקת קריסת הלקוח באופן לא צפוי. מפני שהבדיקה הקודמת עבדה ציפיתי שגם זו תעבוד אך יש צורך בבדיקה לתוכן ההודעה אם היא ריקה במקרה של התנתקות יזומה וטיפול בכך מצד השרת.

בדיקות נוספות שבוצעו:**1. בדיקת קריסת הלקוח באופן לא צפוי**

- **מטרת הבדיקה:** לבדוק את תגובת השרת לקריסת לקוח.
- **מה בוצע בפועל:** סגרנו את תוכנת הלקוח בצורה פתאומית.
- **תוצאת הבדיקה:** השרת זיהה את הניתוק וסגר את החיבור.
- **בעיות ופתרון:** לא התגלו בעיות.

2. בדיקת קריסת השרת באופן לא צפוי

- **מטרת הבדיקה:** לבדוק את תגובת הלקוח כאשר השרת קורס.
- **מה בוצע בפועל:** הופסק הרצת השרת בזמן פעילות לקוחות.
- **תוצאת הבדיקה:** הלקוחות איבדו חיבור אך לא הוצג להם הניתוק מהמערכת.
- **בעיות ופתרון:** העברת הלקוח למסך שגיאה כאשר לא מתאפשרת יותר תקשורת עם השרת.

3. התחברות לחשבון שכבר מחובר

- **מטרת הבדיקה:** לבדוק כיצד המערכת מגיבה לשני חיבורים במקביל עם אותו שם משתמש.

- **מה בוצע בפועל:** בוצעה התחברות כפולה מאותו שם משתמש משני מחשבים שונים.
- **תוצאת הבדיקה:** ההתחברות השנייה נחסמה.
- **בעיות ופתרון:** לא התגלו בעיות.

4. ניסיון התחברות לשרת כשהוא סגור

- **מטרת הבדיקה:** לבדוק האם הלקוח יודע לזהות ששרת אינו פעיל.
- **מה בוצע בפועל:** הופעלה תוכנת לקוח כששרת כבוי.
- **תוצאת הבדיקה:** לא הוצגה הודעת שגיאה מתאימה.
- **בעיות ופתרון:** היה צורך להוספת לוגיקה נוספת מצד הלקוח בשביל שתוצג הודעה מתאימה במצב זה, טיפול בשגיאת ConnectionRefusedError.

5. ניסיון להזרקת SQL

- **מטרת הבדיקה:** לבדוק אם קיימת חסינות של המערכת מפני SQL Injection.
- **מה בוצע בפועל:** נשלחו שאילתות זדוניות בשדות ההתחברות והרישום.
- **תוצאת הבדיקה:** הקלט לא גרם לפעולה מסוכנת, המערכת נותרה יציבה.
- **בעיות ופתרון:** לא התגלו בעיות, השימוש בפרמטרים עם prepared statements הגן על המערכת.

מדריך למשתמש

עץ קבצים

- קובץ פייתון שרת - server.py
- קובץ פייתון לקוח - client.py
- קובץ פייתון הצפנה - encryption.py
- קובץ פייתון מסד נתונים - db.py
- קובץ פייתון בוט טלגרם - telegram_bot.py
- קובץ פייתון קבועים - constants.py
- קובץ אייקון לפרויקט - watchdog.ico
- תמונת אין וידאו (מוצגת אצל משתמש כאשר אינו מקבל וידאו) - no_video.jpg
- קובץ לוגו הפרויקט - logo.png
- קובץ מסד נתונים (נוצר לאחר הרצת קוד המערכת במידה ולא קיים כבר) - watchdog.db

התקנת המערכת

פירוט הסביבה הנדרשת:

כדי להריץ את המערכת נדרשים הדברים הבאים:

- מערכת הפעלה: Windows 10/11
- חיבור לרשת (LAN או Wi-Fi)
- מצלמת רשת תקינה (בצד לקוח מצלמה)
- התקנת פייתון

להפעלת המערכת נדרש להתקין את הכלים והתוכנות הבאות:

- פייצ'ארם (להפעלה נוחה, ניתן גם להריץ דרך שורת הפקודה אך פחות מומלץ)
- פייתון

מיקומי קבצים

כל קבצי הפרויקט ממוקמים באותה תיקייה ראשית בשם watchdog. כל קובץ יש להריץ לפי תפקידו. את server.py ו-telegram_bot.py יש להריץ פעם אחת בצד השרת, ואת client.py בכל

מחשב שמתחבר.

נתונים התחלתיים:

- טוקן טלגרם עבור שליחת הודעות התרעה נדרש ונמצא בקובץ constants.py. יש ליצור בוט טלגרם ולהזין את הטוקן בתצורת הקובץ.
- מפתחות הצפנה ופענוח בשביל הצפנת ה-RSA, נמצא בקובץ constants.py.

רשת:

- כל הרכיבים (השרת, המשתמשים, המצלמות) חייבים להיות מחוברים לאותה רשת פנימית.
- יש לוודא שהפורטים בהם השרת משתמש אינם חסומים בחומת האש של המחשב או הנתב.

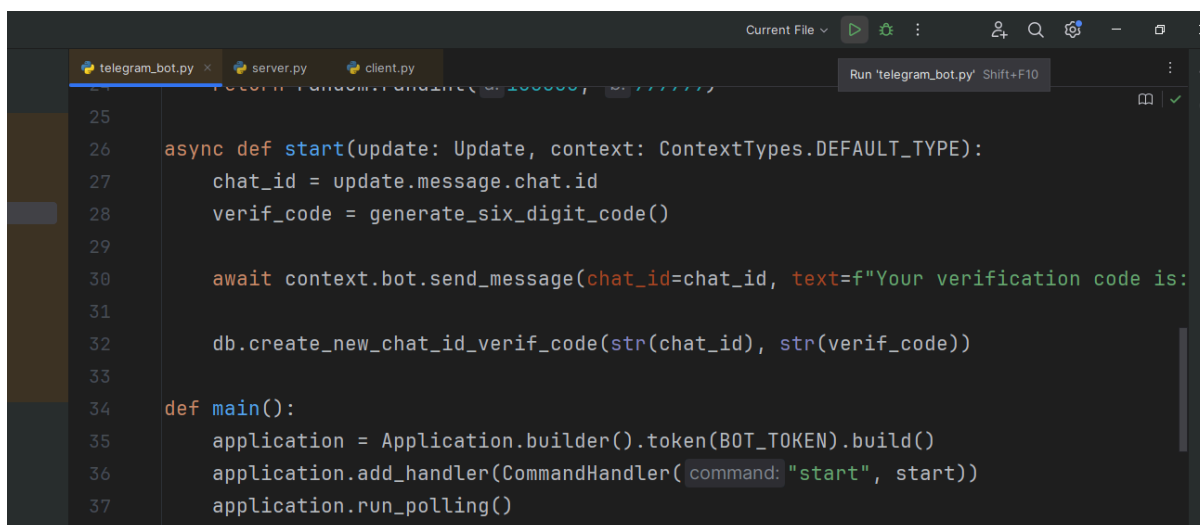
ארכיטקטורה מינימלית נדרשת:

הארכיטקטורה המינימלית שנדרשת היא מחשב שיריץ את השרת ואת בוט הטלגרם, ניתן להתחבר לשרת ממחשבים נוספים באותה שרת ואפילו ניתן מאותו מחשב של השרת.

אופן הפעלת המערכת

מצד השרת:

מצד השרת יש להריץ את שני הקבצים הבאים: telegram_bot.py ו-server.py. ניתן להריצם בסביבת הפייצ'ארם על ידי לחיצה על המשולש הירוק המסומן בצילומי המסך או על ידי צירוף לחיצה על שני המקשים הבאים בו זמנית: Shift+F10.



```

25
26 async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
27     chat_id = update.message.chat.id
28     verif_code = generate_six_digit_code()
29
30     await context.bot.send_message(chat_id=chat_id, text=f"Your verification code is:
31
32     db.create_new_chat_id_verif_code(str(chat_id), str(verif_code))
33
34 def main():
35     application = Application.builder().token(BOT_TOKEN).build()
36     application.add_handler(CommandHandler(command="start", start))
37     application.run_polling()

```

```

1 import socket, select
2 import sqlite3
3 import threading
4 import constants
5 import db
6 import struct
7 from encryption import *
8 import telegram_bot
9
10
11 class Client:
12     def __init__(self, client_socket, name):
13         self.client_socket = client_socket
14         self.name = name

```

מצד הלקוח:

מצד הלקוחות (משתמשי המערכת והמצלמות שרשומות במערכת) יש להריץ את קובץ הקליינט (לקוח).

```

17 class App(CTk):
18     def __init__(self):
19         self.register_frame.grid(row=0, column=0, padx=20, pady=20, sticky="nsew")
20
21         self.user_frame = UserFrame(app=self)
22         self.user_frame.grid(row=0, column=0, padx=20, pady=20, sticky="nsew")
23
24         self.camera_selection_frame = CameraSelectionFrame(app=self)
25         self.camera_selection_frame.grid(row=0, column=0, padx=20, pady=20, sticky="nsew")
26
27         self.camera_frame = CameraFrame(app=self)
28         self.camera_frame.grid(row=0, column=0, padx=20, pady=20, sticky="nsew")
29
30         # The frame we are starting with
31         self.show_login_frame()
32
33     def close_app(self):
34         print("Closing server connection and exiting...")
35         try:
36             client_socket.close()
37         except Exception as e:

```

לאחר הרצת קובץ הלקוח נפתח מסך ההתחברות, אם קיים חשבון מצלמה/משתמש אז על בעל החשבון להזין את שם החשבון ואת הסיסמה וללחוץ על כפתור ההתחברות.

WATCHDOG - Login



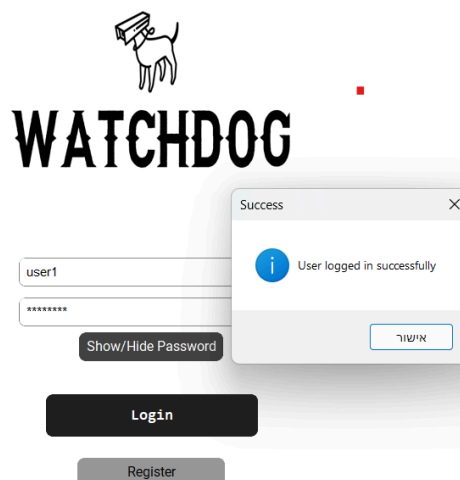
Show/Hide Password

Login

Register

לאחר התחברות מוצלחת לחשבון יועבר הלקוח לתוכנה המיועדת למשתמש/מצלמה בהתאם לסוג החשבון אליו התחבר הלקוח.

WATCHDOG - Login



במידה וללקוח שפתח את התוכנה אין חשבון או שמעוניין ביצירת חשבון נוסף, ילחץ הלקוח על כפתור ההרשמה שנמצא מתחת לכפתור ההתחברות ויובל למסך הבא:



Show/Hide Password

User

Register

Login

כעת על הלקוח לבחור שם וסיסמה לחשבון שיוצר. כמו כן, על הלקוח להחליט האם הוא יוצר חשבון מסוג משתמש או מסוג מצלמה - עליו לעשות זאת על ידי בחירה באפשרות מתפריט האפשרויות שמופיע מעל כפתור ההרשמה למערכת.



Show/Hide Password

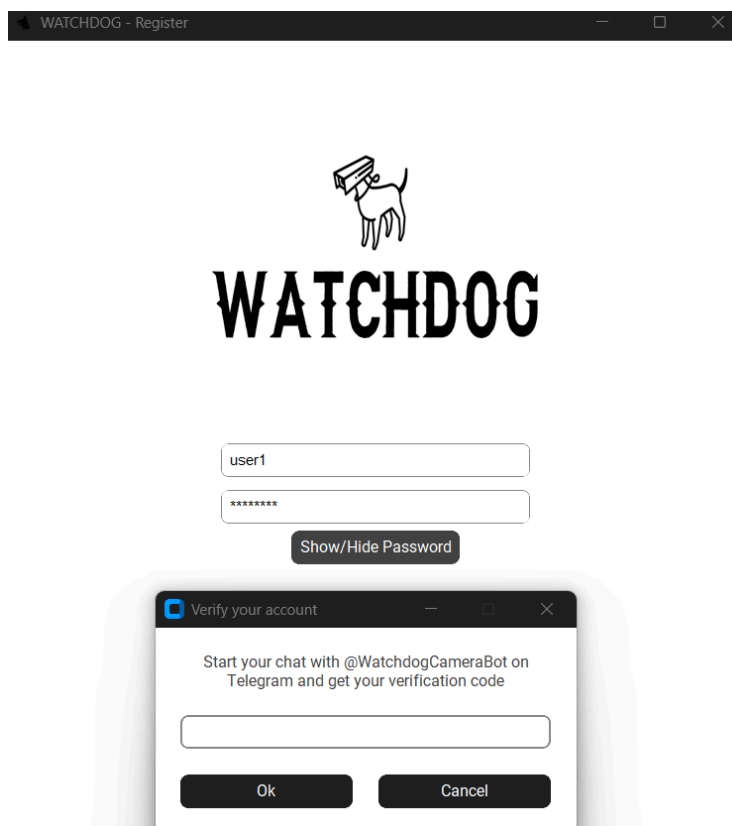
User

Register

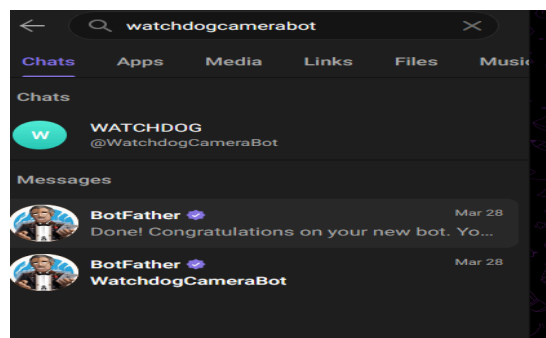
Login

מצד משתמש:

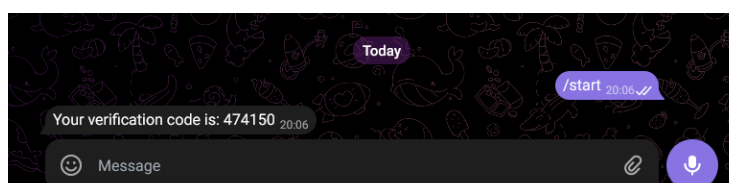
במידה והלקוח ניסה ליצור חשבון מסוג משתמש, יופיע החלון הבא מעל חלון האפליקציה הנוכחי. חשוב לציין שחלון זה מופיע רק כאשר יוצרים חשבון חדש מסוג משתמש, הוא לא מופיע כשיוצרים חשבון מסוג מצלמה וגם לא כאשר מתחברים לחשבון משתמש.

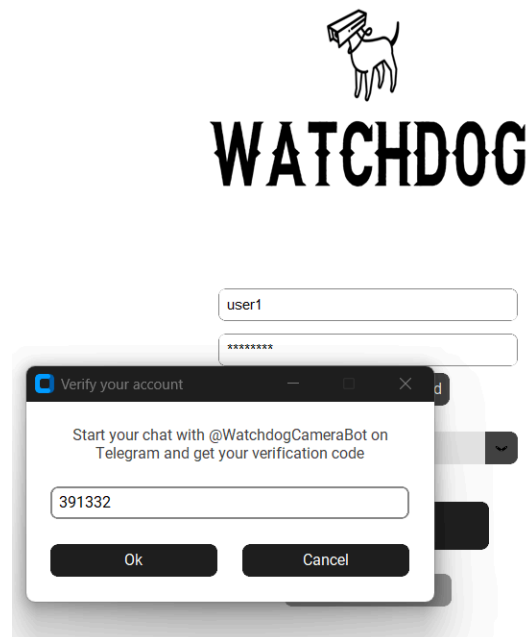


כעת על הלקוח לעקוב אחר ההנחיות שרשומות בחלון שנפתח ולפתוח טלגרם בשביל לקבל קוד אימות להרשמה למערכת. עליו לחפש בחיפוש הטלגרם אחר @WatchdogCameraBot.

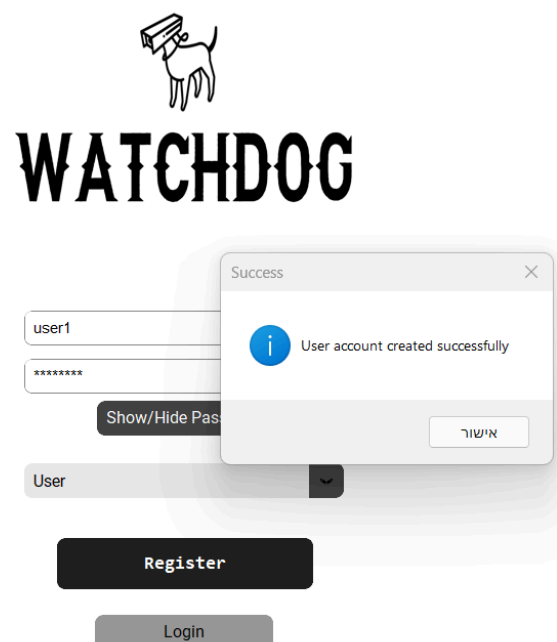


כאשר הלקוח מתחיל ב"שיחתו" עם בוט הטלגרם יופיע בתחתית המסך כפתור כחול שרשום עליו START, על הלקוח ללחוץ על כפתור זה. לאחר לחיצה על כפתור זה יישלח start/ על ידי הלקוח לבוט הטלגרם והוא יקבל בהודעה חזרה קוד אימות בן 6 ספרות. כעת על הלקוח להעתיק את הקוד הנ"ל אל החלון שמבקש אותו בתוכנה וללחוץ על OK.





בתנאי שעומד החשבון החדש בכל התנאים שדרושים ליצירת חשבון משתמש יקבל הלקוח הודעת הצלחה בנוגע ליצירת חשבון המשתמש החדש ויוכל ללחוץ על אישור בשביל לפתוח את התוכנה.



כעת נפתח מסך המשתמש לשליטה בתוכנה. אילו הייתה איזושהי מצלמה מחוברת למערכת, המשתמש היה אוטמטית מתחבר למצלמה בעלת ה-ID הנמוך ביותר שמחוברת למערכת. מפני ששום מצלמה ללא מחוברת למערכת רשום למשתמש שאינו מביט בשום מצלמה ועליו לנסות להחליף מצלמות בשביל שיוכל לראות וידאו.

WATCHDOG - User

No camera - try to switch cameras



No Video Available

Screenshot

Switch camera

Mirror footage

Subscribe to camera

Alert all camera subscribers

אם המשתמש לוחץ על כפתור החלפת המצלמה ואין שום מצלמה מחוברת אז לא יקרה כלום. אבל במידה והתחברה מצלמה למערכת בזמן זה ולחץ המשתמש על החלפת מצלמה אז התמונה תוחלף בוידאו המצלמה אליה החליף והמשתמש יוכל כעת לצפות בוידאו המצלמה הנוכחית ולהשתמש במערכת או לנסות להחליף למצלמה אחרת.

WATCHDOG - User

Camera: cam1 ID: 1



Screenshot

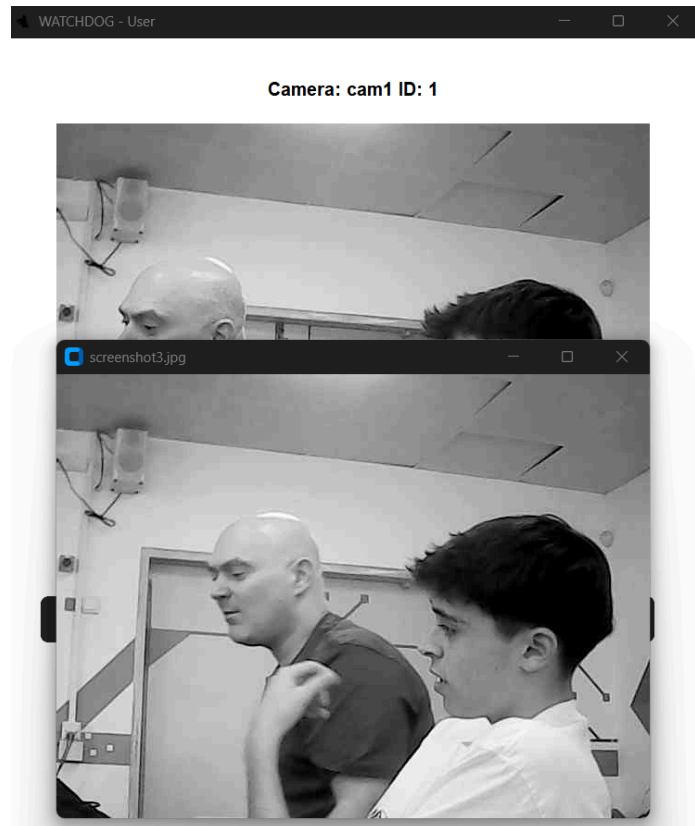
Switch camera

Mirror footage

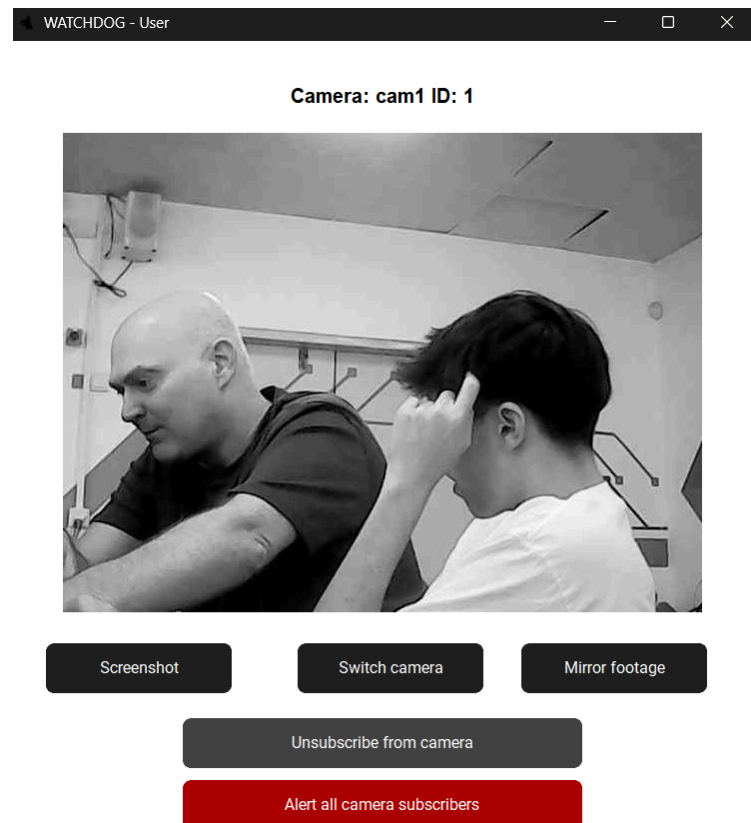
Subscribe to camera

Alert all camera subscribers

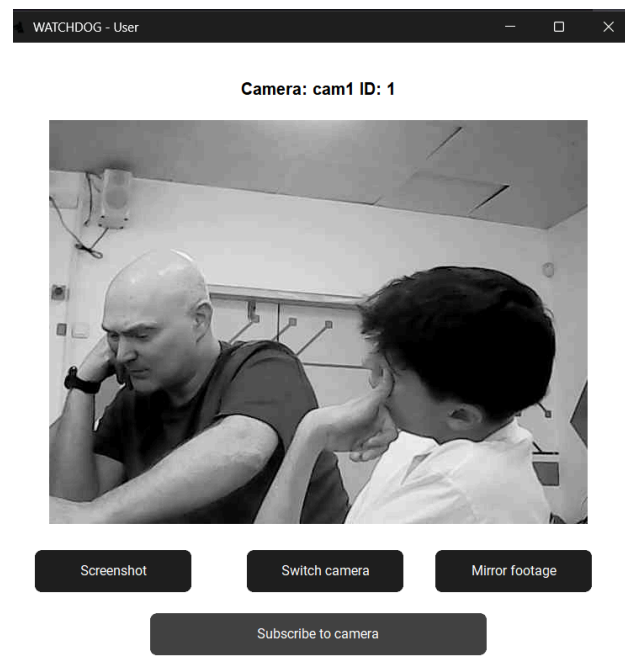
אחת האפשרויות שיש למשתמש היא צילום תמונה של התמונה הנוכחית שמוצגת אצל המשתמש, תמונה זו תישמר בקובץ מסוג jpg בתיקיית הפרויקט.



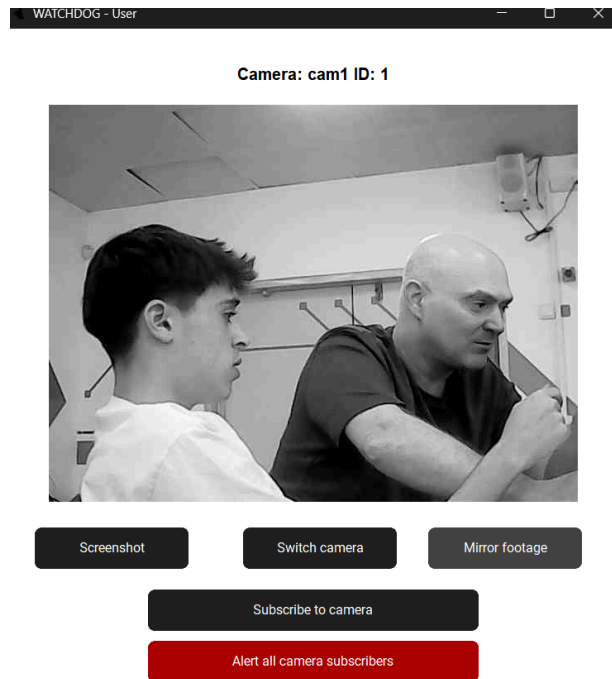
אפשרות נוספת שיש למשתמש היא הרשמה למצלמה בה הוא צופה. כל משתמש יכול להיות רשום לכמה מצלמות שירצה ואם משתמש רשום למצלמה מסוימת אז יקבל המשתמש עדכונים בטלגרם בכל פעם שמשתמש אחר בוחר להתריע לגבי המתרחש במצלמה זו.



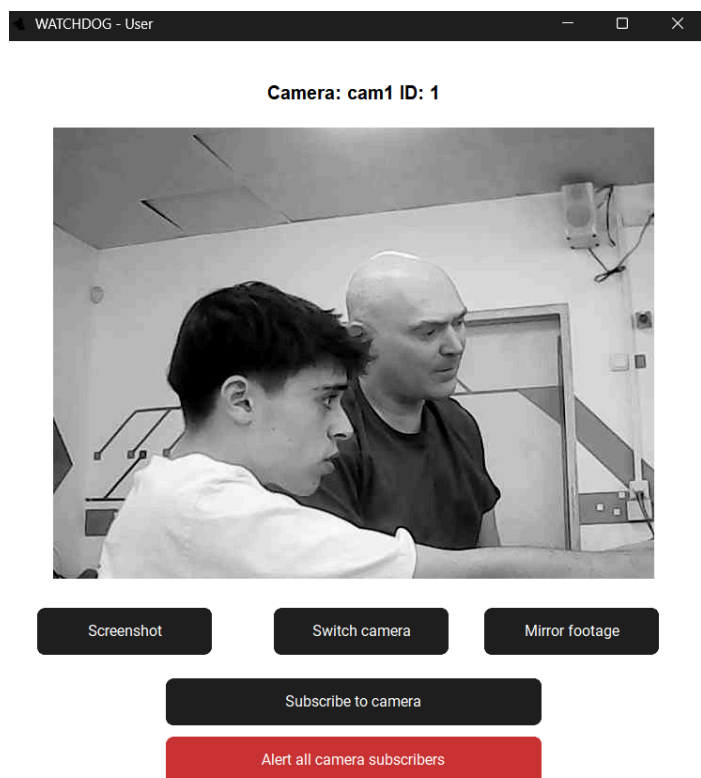
לאחר שמשתמש נרשם למצלמה יש באפשרותו לבטל את ההרשמה למצלמה על ידי לחיצה על אותו הכפתור, שעת רשום עליו שזה כפתור ביטול הרשמה למצלמה.



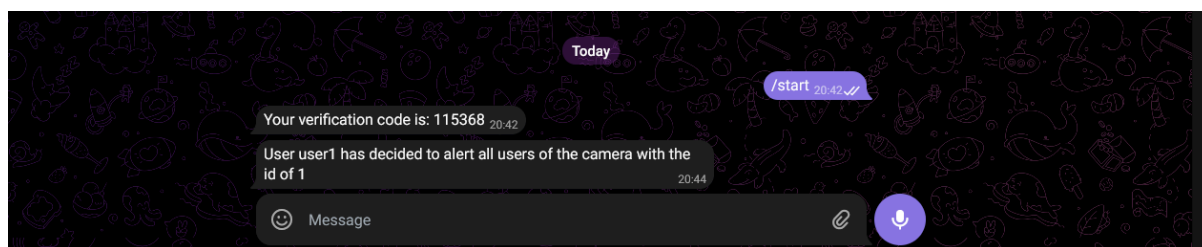
למשתמש גם אפשרות ללחוץ על כפתור לשיקוף הוידאו אותו מקבל מהמצלמה. הוידאו משוקף רק אצל המשתמש וכל שאר המשתמשים שמחוברים לאותה מצלמה יראו את הוידאו כרגיל.



לבסוף, למשתמש יש את האפשרות ללחוץ על כפתור ההתרעה ולשלוח הודעה מתריעה בטלגרם לכל המשתמשים הרשומים לאותה מצלמה דרך הטלגרם.



להלן דוגמה להודעה שנשלחה למשתמש הרשום למצלמה בעלת ה-ID 1 בטלגרם כאשר משתמש בשם user1 לחץ על כפתור ההתרעה.



מצד מצלמה:

כעת יוצג איך עובדת התוכנה כאשר יוצרים חשבון מסוג מצלמה במקום מסוג משתמש. על הלקוח לבחור בתפריט האפשרויות מעל כפתור ההרשמה בחשבון מסוג מצלמה וללחוץ על יצירת חשבון.



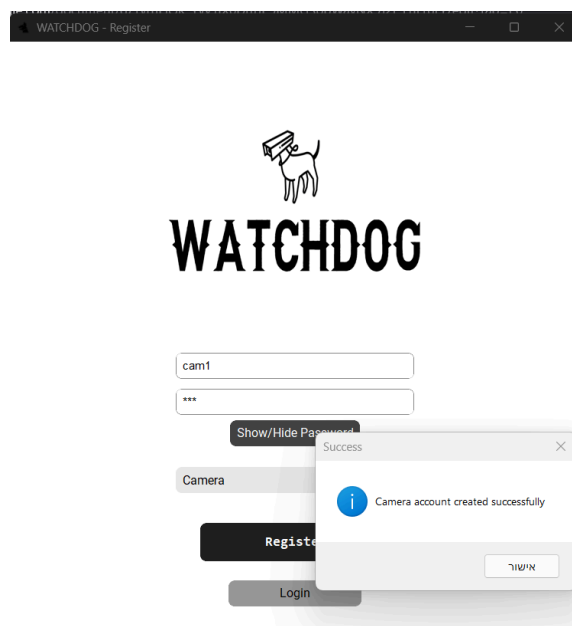

Show/Hide Password

Camera

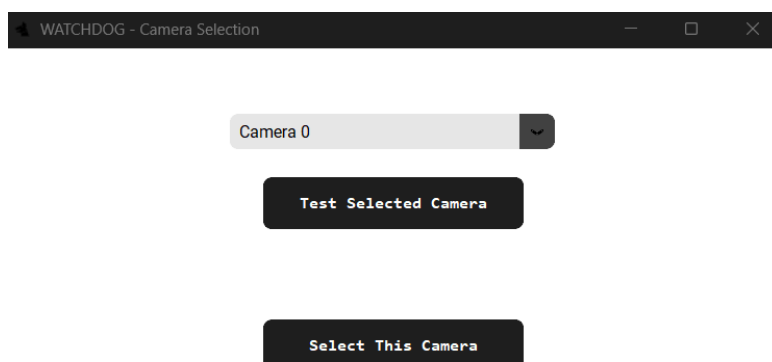
Register

Login

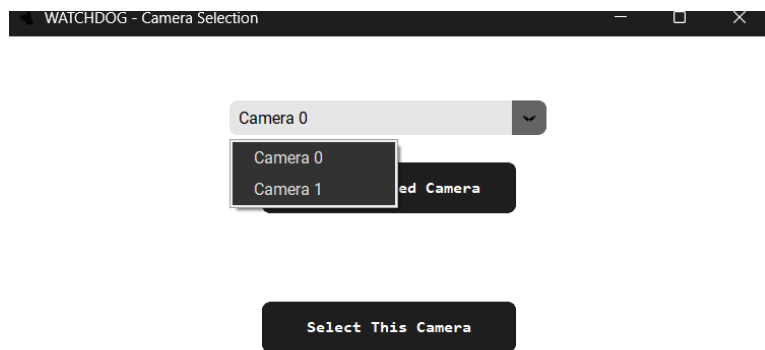
בתנאי שעומד החשבון החדש בכל התנאים הדרושים ליצירת חשבון מצלמה יקבל הלקוח הודעת הצלחה בנוגע ליצירת חשבון המשתמש החדש ויוכל ללחוץ על אישור בשביל לפתוח את התוכנה.



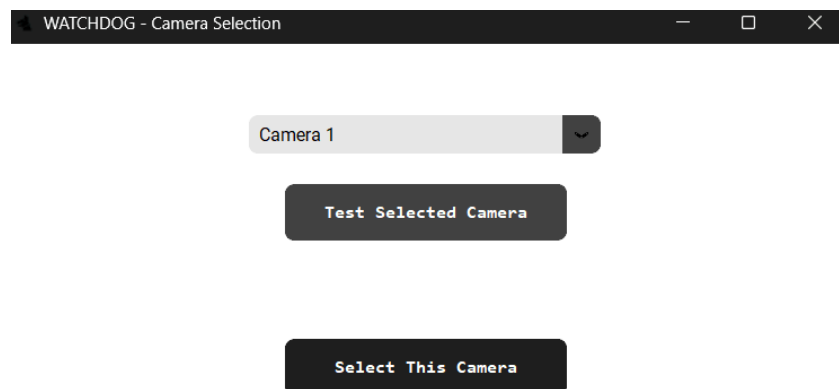
כעת יפתח לבעל חשבון המצלמה תפריט בחירת המצלמה אותה ישדר למערכת, בתפריט זה יוכל לבדוק את המצלמות השונות המחוברות למחשב ולא תפוסות על ידי תוכנות אחרות בשביל לשדר למערכת.



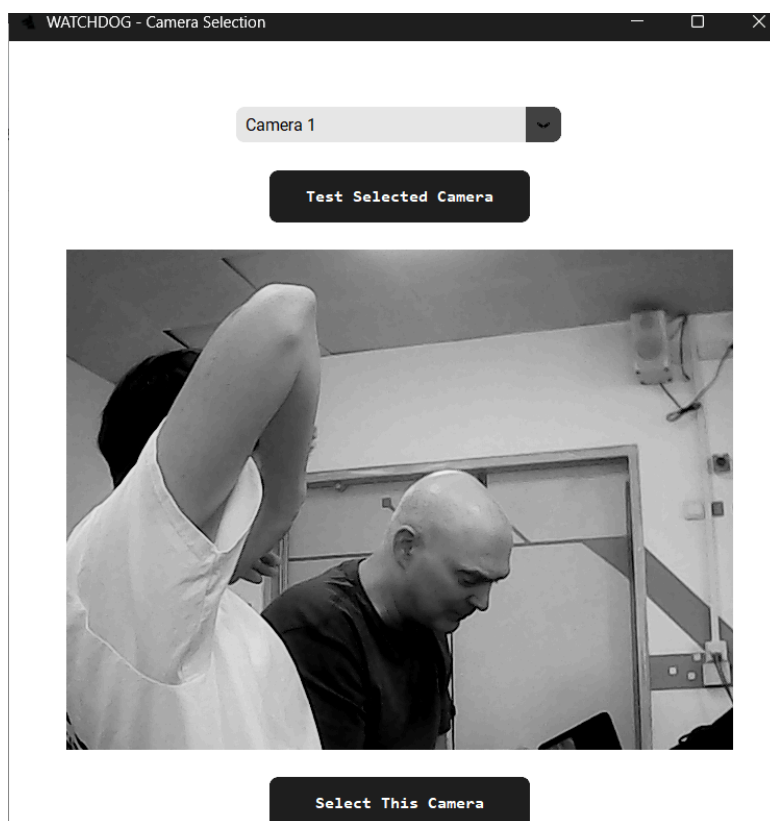
על הלקוח לבחור במצלמה אותה רוצה לבדוק בתפריט האפשרויות שמופיע בראש התוכנה, גם אם לא רוצה לבדוק אם עובדת המצלמה לפני תחילת שידורה לשרת המצלמה שיבחר בתפריט תהיה המצלמה אותה ישדר.



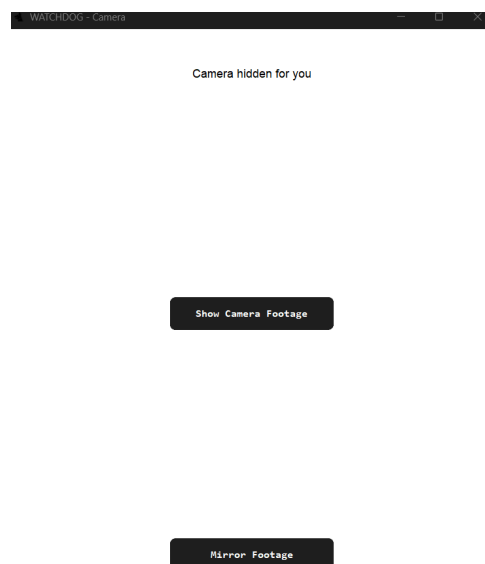
כעת על הלקוח ללחוץ על כפתור בדיקת המצלמה במידה ורוצה לבחון אותה, שלב זה אינו חובה ובאפשרות הלקוח לבחור את המצלמה המוצגת למעלה בלי לבדוק אותה לפני.



כעת הלקוח יכול לראות כיצד יראה הוידאו במצלמה במידה ויחליט לשדר אותה לשרת למשתמשים אשר ישתמשו במערכת, באפשרות הלקוח גם לבחון מצלמות אחרות לפני שיחליט באיזו מצלמה ישתמש. לאחר מכן על הלקוח ללחוץ על בחירת מצלמה זו.



כעת לאחר שבחר הלקוח במצלמה מסוימת, היא משודרת לשרת והמשתמשים במערכת יכולים לצפות בה. המצלמה מוסתרת למפעיל המצלמה בצורה אוטומטית בשביל לחסוך משאבים במחשבו. באפשרות בעל המצלמה להפעיל את וידאו המצלמה לצפייה אצלו וגם לשקף את התמונה אצל כל הלקוחות שמחוברים למצלמה זו.



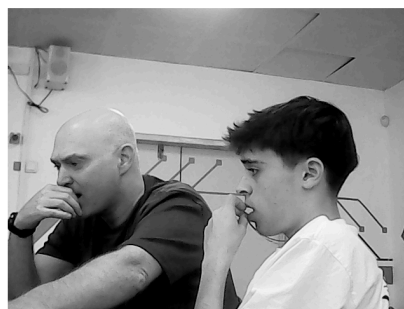
במידה וילחץ מפעיל המצלמה על להראות את וידאו המצלמה במסך שלו, יוכל לראות את הוידאו שרואים המשתמשים שמחוברים למצלמה זו דרך השרת.



Hide camera footage

Mirror Footage

מפעיל המצלמה גם יכול לשקף את הוידאו והוא ישוקף לכל המשתמשים שצופים בוידאו זה (אם המשתמשים הפעילו שיקוף אצלם אז יתבצע שיקוף כפול, ולמעשה יתבטל).



Hide camera footage

Mirror Footage

לבסוף, מפעיל המצלמה יכול גם להפסיק לצפות בוידאו של המצלמה שלו בכל רגע ולהפעיל וידאו זה שוב במידת הצורך.



Camera hidden for you

Show camera footage

Mirror Footage

רפלקציה / סיכום אישי

לאורך העבודה על פרויקט זה עברתי תהליך משמעותי הן מבחינה מקצועית בתור מתכנת ומפתח. בתחילת הדרך היה לי רעיון כללי בלבד ולא הייתה לי הבנה מלאה על הדרך הארוכה שצריך לעבור כדי להוציא אותו לפועל. כל שלב נוסף שנבנה בפרויקט ועבד היה מבחינתי הצלחה שהובילה להבנה רחבה יותר של עולם התכנות והפיתוח.

האתגרים העיקריים כללו טיפול בחיבורים לא יציבים או בקריסות פתאומיות של שרת או לקוח, והטמעה של הצפנה אמינה ומהירה מספיק בשביל לא להפריע למהלך התוכנית.

אתגרים נוספים כללו שימוש נכון בבסיס נתונים, שמירה על קוד נקי, ובדיקת תרחישים שונים (מקרי קצה). בכל פעם שנתקלתי בקושי, חיפשתי מקורות ברשת, שאלתי חברים, ולפעמים גם עם מורים. לעיתים הפתרון לא היה טכני בלבד, אלא היה צורך לחשוב על הדרך הנכונה לגשת לבעיה מבחינת מבנה קוד או עיצוב לוגי. במהלך הפרויקט למדתי בעצמי נושאים רבים שמעולם לא נגעת בהם קודם: כיצד עובדת הצפנת AES, איך עובדים עם API של טלגרם וכיצד לכתוב קוד עמיד בפני תקלות וחסין לקריסות.

מה שאני לוקח איתי להמשך הוא היכולת לפרק בעיה מורכבת לחלקים קטנים וברורים, ולחשוב איך אני רוצה לכתוב את הקוד לפני שאני כותב אותו. כמו כן, למדתי לעבוד בצורת דיבוג יעילה, ולהשתמש בכלים כמו הדפסות בחלקים מסוימים של הקוד, בדיקות של חלקי קוד בנפרד, וכתובת בדיקות קצה. אם הייתי עושה את הפרויקט מחדש, הייתי משקיע יותר זמן בשלבי התכנון המוקדם. היו נקודות בהן הייתי צריך לכתוב מחדש חלקים שלמים בגלל תכנון לא מעמיק מההתחלה.

אם היו לי משאבים נוספים, הייתי משלב בינה מלאכותית במערכת לזיהוי סכנות בזמן אמת במצלמות האבטחה ועוד מגוון פיצ'רים שונים לנוחות המשתמש.

שאלות שעלו לי תוך כדי העבודה:

- האם יש דרך יותר טובה לעשות את מה שאני מתכנן לעשות?
- האם שיטת ההצפנה בה השתמשתי מתאימה להעברת מידע מהסוג הזה?
- האם הקוד שכתבתי ברור ומובן לי ולקורא חיצוני?

תודות:

אני רוצה להודות לצוות מורי הסייבר שעזרו לי על ידי סיפוק מענה לשאלות ונתנו לי להתייעץ איתם בנוגע לדברים שרציתי להוסיף לפרויקט, להוריי שתמכו בי במהלך העבודה על הפרויקט והיו מבינים ומכילים לגבי הלחץ שחווייתי עם העמידה בזמנים, למקורות מידע באינטרנט שסיפקו תשובות לשאלות טכניות רבות ולחבריי בסייבר 1 על אווירת העבודה הטובה בכיתה (תודה בפרט לברק שעזר לי בחלקים שונים בפרויקט ובספר ונתן לי להתייעץ איתו וגם לאורי שהסכים להיות מצולם בתמונות במדריך למשתמש).

ביבליוגרפיה

- ויקיפדיה. Transmission Control Protocol. מתוך ויקיפדיה.
https://he.wikipedia.org/wiki/Transmission_Control_Protocol
- ויקיפדיה. RSA. מתוך ויקיפדיה.
<https://he.wikipedia.org/wiki/RSA>
- ויקיפדיה. AES. מתוך ויקיפדיה.
<https://he.wikipedia.org/wiki/AES>
- ZeroFruit. What is AES? Step-by-step. מתוך מדיום.
<https://zerofruit-web3.medium.com/what-is-aes-step-by-step-fcb2ba41bb20>
- ויקיפדיה. Rijndael S-box. מתוך ויקיפדיה
https://en.wikipedia.org/wiki/Rijndael_S-box
- Python Software Foundation. socket — Low-level networking interface. מתוך תיעוד שפת הפיתון.
<https://docs.python.org/3/library/socket.html>
- Python Software Foundation. struct — Interpret bytes as packed binary data. מתוך תיעוד שפת הפיתון.
<https://docs.python.org/3/library/struct.html>
- Python Cryptographic Authority. Symmetric encryption — Cryptography documentation.
<https://cryptography.io/en/latest/hazmat/primitives/symmetric-encryption>
- Telegram. Telegram Bot API. מתוך טלגרם.
<https://core.telegram.org/bots/api>
- python-telegram-bot Developers. python-telegram-bot documentation.
<https://docs.python-telegram-bot.org/en/stable>
- Telethon Developers. Telethon documentation.
<https://docs.telethon.dev/en/stable>
- Pyrogram Developers. Pyrogram documentation.
<https://docs.pyrogram.org>

נספחים

קוד הלקוח:

```
from customtkinter import *
import socket
import threading
from win32api import MessageBox
from win32con import MB_OK, MB_ICONINFORMATION, MB_ICONERROR
import constants
from encryption import *
import os
import win32file
import cv2
from PIL import Image, ImageTk
import struct
import time
import numpy

class App(CTk):
    def __init__(self):
        super().__init__()
        self.iconbitmap("watchdog.ico")
        self.geometry("600x700")
        self.configure(fg_color="white")
        self.grid_rowconfigure(0, weight=1)
        self.grid_columnconfigure(0, weight=1)
        self.protocol("WM_DELETE_WINDOW", self.close_app)

        self.login_frame = LoginFrame(app=self)
        self.login_frame.grid(row=0, column=0, padx=20, pady=20,
sticky="nsew")

        self.register_frame = RegisterFrame(app=self)
        self.register_frame.grid(row=0, column=0, padx=20, pady=20,
sticky="nsew")

        self.user_frame = UserFrame(app=self)
        self.user_frame.grid(row=0, column=0, padx=20, pady=20,
sticky="nsew")

        self.camera_selection_frame = CameraSelectionFrame(app=self)
        self.camera_selection_frame.grid(row=0, column=0, padx=20,
pady=20, sticky="nsew")
```

```
self.camera_frame = CameraFrame(app=self)
self.camera_frame.grid(row=0, column=0, padx=20, pady=20,
sticky="nsew")

self.exception_frame = ExceptionFrame(app=self)
self.exception_frame.grid(row=0, column=0, padx=20, pady=20,
sticky="nsew")

# The frame we are starting with
self.show_login_frame()

def close_app(self):
    try:
        client_socket.close()
        self.destroy()
    finally:
        exit(0)

def show_login_frame(self):
    self.title("WATCHDOG - Login")
    self.login_frame.lift()

def show_register_frame(self):
    self.title("WATCHDOG - Register")
    self.register_frame.lift()

def show_user_frame(self):
    self.title("WATCHDOG - User")
    self.user_frame.start_receiving_footage()
    self.user_frame.lift()

def show_camera_selection_frame(self):
    self.title("WATCHDOG - Camera Selection")
    self.camera_selection_frame.update_camera_indexes()
    self.camera_selection_frame.lift()

def show_camera_frame(self, camera_index=0):
    self.title("WATCHDOG - Camera")
    self.camera_frame.start_sending_camera_footage(camera_index)
    self.camera_frame.lift()

def show_exception_frame(self, error):
    self.title("WATCHDOG - Error")
    self.exception_frame.update_error_msg(error)
    self.exception_frame.lift()
```

```
class ExceptionFrame(CTkFrame):
    def __init__(self, app):
        super().__init__(app)
        self.configure(fg_color="white")

        self.logo_label = CTkLabel(self, text="",
image=CTkImage(light_image=Image.open("logo.png"),
dark_image=Image.open("logo.png"), size=(300, 300)))
        self.logo_label.pack()

        self.error_msg = CTkLabel(self, text="Error occurred - Proceed by
closing the app", text_color="black", font=("Segoe UI", 20, "bold"))
        self.error_msg.pack()

    def update_error_msg(self, error):
        self.error_msg.configure(text=f"{error} - Proceed by closing the
app")

class LoginFrame(CTkFrame):
    def __init__(self, app):
        super().__init__(app)
        self.app = app
        self.configure(fg_color="white")

        # Configure grid columns for spacing
        self.grid_columnconfigure(0, weight=1)
        self.grid_columnconfigure(1, weight=0)
        self.grid_columnconfigure(2, weight=1)

        self.logo_label = CTkLabel(self, text="",
image=CTkImage(light_image=Image.open("logo.png"),
dark_image=Image.open("logo.png"), size=(300, 250)))
        self.logo_label.grid(row=0, column=0, columnspan=3, pady=(30,
20), sticky="n")

        self.username_entry = CTkEntry(self, font=("Arial", 12),
text_color="black", placeholder_text="Enter account name", width=250,
fg_color="white", border_color="light gray", border_width=1,
corner_radius=6)
        self.username_entry.grid(row=1, column=0, columnspan=3, pady=5)
        self.username_entry.bind("<Key>", lambda x: "break" if x.keysym
== "space" else None)

        self.password_entry = CTkEntry(self, font=("Arial",
```

```

12),text_color="black", placeholder_text="Enter password", show="*",
width=250, fg_color="white", border_color="light gray", border_width=1,
corner_radius=6)
    self.password_entry.grid(row=2, column=1, pady=5,)
    self.password_entry.bind("<Key>", lambda x: "break" if x.keysym
== "space" else None)

    self.show_password_button = CTkButton(self, text="Show/Hide
Password", width=20, fg_color="gray26", hover_color="gray",
text_color="white", command=lambda:
self.password_entry.configure(show="") if
self.password_entry.cget("show") == "*" else
self.password_entry.configure(show="*"))
    self.show_password_button.grid(row=3, column=1, pady=(0, 10))

    self.login_button = CTkButton(self, text="Login",
command=self.login_to_account_thread_start, font=("Consolas", 14,
"bold"), height=40, width=200, fg_color="black", hover_color="gray26",
text_color="white")
    self.login_button.grid(row=4, column=0, colspan=3, pady=20)

    self.register_button = CTkButton(self, text="Register",
command=self.app.show_register_frame, fg_color="light gray",
hover_color="light gray", text_color="black")
    self.register_button.grid(row=5, column=0, colspan=3)

    def login_to_account_thread_start(self):
        threading.Thread(target=self.login_to_account).start()

    def login_to_account(self):
        try:
            client_socket.send(rsa_encrypt(constants.client_to_server_public_key,
"login").encode())
            client_socket.recv(1024)

            client_socket.send(rsa_encrypt(constants.client_to_server_public_key,f"{
self.username_entry.get()} {self.password_entry.get()}").encode())
            answer = rsa_decrypt(constants.server_to_client_private_key,
client_socket.recv(1024).decode())

            if "Wrong password" in answer:
                MessageBox(0, "Wrong password", 'Error', MB_OK |
MB_ICONERROR)
            elif "No such username" in answer:

```

```

        MessageBox(0, "No such username", 'Error', MB_OK |
MB_ICONERROR)
    elif "already logged in" in answer:
        MessageBox(0, answer, 'Error', MB_OK | MB_ICONERROR)
    elif "Too many login attempts" in answer:
        self.app.show_exception_frame("Too many login attempts")
    else:
        MessageBox(0, answer, 'Success', MB_OK |
MB_ICONINFORMATION)

        if answer.split()[0] == "User":
            self.app.show_user_frame()
        elif answer.split()[0] == "Camera":
            self.app.show_camera_selection_frame()

    except (BrokenPipeError, ConnectionResetError, socket.timeout,
OSError):
        self.app.show_exception_frame("Lost connection to server")

class RegisterFrame(CTkFrame):
    def __init__(self, app):
        super().__init__(app)
        self.app = app
        self.configure(fg_color="white")

        # Configure grid columns for spacing
        self.grid_columnconfigure(0, weight=1)
        self.grid_columnconfigure(1, weight=0)
        self.grid_columnconfigure(2, weight=1)

        self.logo_label = CTkLabel(self, text="",
image=CTkImage(light_image=Image.open("logo.png"),
dark_image=Image.open("logo.png"), size=(300, 250)))
        self.logo_label.grid(row=0, column=0, columnspan=3, pady=(30,
20), sticky="n")

        self.username_entry = CTkEntry(self, font=("Arial", 12),
text_color="black", placeholder_text="Enter account name", width=250,
fg_color="white", border_color="light gray", border_width=1,
corner_radius=6)
        self.username_entry.grid(row=1, column=0, columnspan=3, pady=5)
        self.username_entry.bind("<Key>", lambda x: "break" if x.keysym
== "space" else None)

        self.password_entry = CTkEntry(self, font=("Arial", 12),

```

```

text_color="black", placeholder_text="Enter password", show="*",
width=250, fg_color="white", border_color="light gray", border_width=1,
corner_radius=6)
    self.password_entry.grid(row=2, column=1, pady=5)
    self.password_entry.bind("<Key>", lambda x: "break" if x.keysym
== "space" else None)

    self.show_password_button = CTkButton(self, text="Show/Hide
Password", width=20, fg_color="gray26", hover_color="gray",
text_color="white", command=lambda: self.password_entry.configure(
show="") if self.password_entry.cget("show") == "*" else
self.password_entry.configure(show="*"))
    self.show_password_button.grid(row=3, column=1, pady=(0, 10))

    self.account_type = CTkOptionMenu(self, values=["User",
"Camera"], width=250, fg_color="gray", button_color="gray26",
button_hover_color="gray", text_color="black")
    self.account_type.grid(row=4, column=1, pady=10)

    self.register_button = CTkButton(self, text="Register",
command=self.register_account_thread_start, font=("Consolas", 14,
"bold"), height=40, width=200, fg_color="black", hover_color="gray26",
text_color="white")
    self.register_button.grid(row=5, column=0, columnspan=3, pady=20)

    self.login_button = CTkButton(self, text="Login",
command=self.app.show_login_frame, fg_color="light gray",
hover_color="light gray", text_color="black")
    self.login_button.grid(row=6, column=0, columnspan=3)

    def register_account_thread_start(self):
        threading.Thread(target=self.register_account).start()

    def register_account(self):
        try:
            dialog = None

client_socket.send(rsa_encrypt(constants.client_to_server_public_key,
"register").encode())
            client_socket.recv(1024)

client_socket.send(rsa_encrypt(constants.client_to_server_public_key,f"{
self.username_entry.get()} {self.password_entry.get()}
{self.account_type.get()}").encode())

```

```

        answer = rsa_decrypt(constants.server_to_client_private_key,
client_socket.recv(1024).decode())

        if answer == "Attempted username is taken":
            MessageBox(0, answer, 'Error', MB_OK | MB_ICONERROR)

        elif answer == "verification code needed":
            right_code_sent = False
            while not right_code_sent:
                dialog = CtkInputDialog(title="Verify your account",
text="Start your chat with @WatchdogCameraBot on Telegram and get your
verification code", text_color="black", fg_color="white",
button_fg_color="black", button_hover_color="gray26",
button_text_color="white", entry_fg_color="white",
entry_border_color="light gray", entry_text_color="black")

                client_socket.send(rsa_encrypt(constants.client_to_server_public_key,
f"{dialog.get_input()}").encode())
                ans =
rsa_decrypt(constants.server_to_client_private_key,
client_socket.recv(1024).decode())
                if ans == "No such verification code":
                    MessageBox(0, "Wrong verification code", 'Error',
MB_OK | MB_ICONERROR)
                elif ans == "User account created successfully":
                    right_code_sent = True
                    MessageBox(0, ans, 'Success', MB_OK |
MB_ICONINFORMATION)

            self.app.show_user_frame()

        elif answer == "Camera account created successfully":
            MessageBox(0, answer, 'Success', MB_OK |
MB_ICONINFORMATION)
            self.app.show_camera_selection_frame()

    except (BrokenPipeError, ConnectionResetError, socket.timeout,
OSError):
        try:
            dialog.destroy()
        finally:
            pass

        self.app.show_exception_frame("Lost connection to server")

```

```

class UserFrame(CTkFrame):
    def __init__(self, app):
        super().__init__(app)
        self.app = app
        self.configure(fg_color="white")

        self.grid_columnconfigure(0, weight=1)
        self.grid_columnconfigure(1, weight=1)
        self.grid_columnconfigure(2, weight=1)

        self.camera_title = CTkLabel(self, text="No camera - try to
switch cameras", text_color="black", font=("Arial", 16, "bold"))
        self.camera_title.grid(row=0, column=0, columnspan=3, pady=(10,
15))

        self.receiving_footage = False
        self.receiving_footage_thread =
threading.Thread(target=self.receive_camera_footage)

        self.image_to_display_in_numpy_array =
numpy.array(Image.open("no_video.jpg"))
        self.camera_encryption_key = None

        self.camera_label = CTkLabel(self, text="")
        self.camera_label.grid(row=1, column=0, columnspan=3, pady=(0,
20))

        self.screenshot_button = CTkButton(self, text="Screenshot",
command=self.screenshot_start_thread, width=150, height=40,
fg_color="black", hover_color="gray26", text_color="white")
        self.screenshot_button.grid(row=2, column=0, padx=10, pady=5,
sticky="w")

        self.switch_camera_button = CTkButton(self, text="Switch camera",
command=self.switch_camera_request_start_thread, width=150,
height=40, fg_color="black", hover_color="gray26", text_color="white")
        self.switch_camera_button.grid(row=2, column=1, padx=10, pady=5,
sticky="e")

        self.mirrored = False
        self.mirror_button = CTkButton(self, text="Mirror footage",
command=self.mirror_footage, width=150, height=40, fg_color="black",
hover_color="gray26", text_color="white")
        self.mirror_button.grid(row=2, column=2, padx=10, pady=5)

```

```

        self.subscribe_button = CTkButton(self, text="Subscribe to
camera", command=self.sub_start_thread, width=320, height=40,
fg_color="black", hover_color="gray26", text_color="white")
        self.subscribe_button.grid(row=3, column=0, columnspan=3,
pady=(15, 5))
        self.subscribed = None

        self.alert_button = CTkButton(self, text="Alert all camera
subscribers", command=self.alert_start_thread, width=320, height=40,
fg_color="red", hover_color="red", text_color="white")
        self.alert_button.grid(row=4, column=0, columnspan=3, pady=(5,
15))

def start_receiving_footage(self):
    self.switch_camera_request_start_thread()
    self.show_camera_footage()

def mirror_footage(self):
    self.mirrored = not self.mirrored

def screenshot_start_thread(self):
    threading.Thread(target=self.screenshot).start()

def screenshot(self):
    image = Image.fromarray(self.image_to_display_in_numpy_array)

    screenshots_folder = os.getcwd()
    path = os.path.join(screenshots_folder, '*')
    file_name = None

    files = win32file.FindFilesW(path)
    for f in files:
        if ".jpg" in f[8] and "screenshot" in f[8]:
            file_name = f[8]

    if file_name is None:
        file_name = "screenshot0.jpg"
    else:
        file_name = "screenshot" +
str(int(file_name.split('.')[0].split('screenshot')[1]) + 1) + ".jpg"

    image.save(file_name)

    toplevel = CTkToplevel(self)
    toplevel.attributes("-topmost", True)

```

```

        toplevel.title(file_name)

        photo_image = ImageTk.PhotoImage(image=image)
        image_label = CTkLabel(toplevel, text="", image=photo_image)
        image_label.pack()

    def alert_start_thread(self):
        threading.Thread(target=self.alert_camera_subscribers).start()

    def alert_camera_subscribers(self):
        try:

client_socket.send(rsa_encrypt(constants.client_to_server_public_key,
"alert").encode())
            except (BrokenPipeError, ConnectionResetError, OSError):
                self.app.show_exception_frame("Lost connection to server")

    def sub_start_thread(self):
        threading.Thread(target=self.subscribe_to_camera).start()

    def subscribe_to_camera(self):
        try:
            self.subscribed = not self.subscribed

client_socket.send(rsa_encrypt(constants.client_to_server_public_key,
"subscribe").encode())
            if not self.subscribed:
                self.subscribe_button.configure(text="Subscribe to
camera")
            else:
                self.subscribe_button.configure(text="Unsubscribe from
camera")
            except (BrokenPipeError, ConnectionResetError, socket.timeout,
OSError):
                self.app.show_exception_frame("Lost connection to server")

    def switch_camera_request_start_thread(self):
        threading.Thread(target=self.switch_camera_request).start()

    def switch_camera_request(self):
        try:
            self.receiving_footage = False
            while self.receiving_footage_thread.is_alive():
                pass

            trash_data = b""

```

```

        client_socket.settimeout(1)
        try:
            while True:
                trash_data += client_socket.recv(4096)
            except socket.timeout:
                pass
            except (BrokenPipeError, ConnectionResetError, OSError):
                self.app.show_exception_frame("Lost connection to
server")

        client_socket.settimeout(5)

        client_socket.send(rsa_encrypt(constants.client_to_server_public_key,
"switch_camera_request").encode())

        ans = ""
        try:
            ans = rsa_decrypt(constants.server_to_client_private_key,
client_socket.recv(1024).decode()).split()
        except UnicodeDecodeError:
            self.switch_camera_request()

        if ans == ["no_cameras"]:
            self.image_to_display_in_numpy_array =
numpy.array(Image.open("no_video.jpg")) # No cameras
            self.camera_title.configure(text="No camera - try to
switch cameras")
            self.subscribed = None
            self.camera_encryption_key = None
        elif len(ans) == 3:
            self.camera_title.configure(text=f"Camera: {ans[0]} ID:
{ans[1]}")
            self.subscribed = eval(ans[2])
            if not self.subscribed:
                self.subscribe_button.configure(text="Subscribe to
camera")
            else:
                self.subscribe_button.configure(text="Unsubscribe
from camera")

        client_socket.send(rsa_encrypt(constants.client_to_server_public_key,
"aes_key_request").encode()) # confirming receiving and asking for
camera's encryption key
        try:
            self.camera_encryption_key =

```

```

bytes.fromhex(rsa_decrypt(constants.server_to_client_private_key,
client_socket.recv(1024).decode()))
    except UnicodeDecodeError:
        time.sleep(0.5)
        self.switch_camera_request()
        self.receiving_footage = True
        self.receiving_footage_thread =
threading.Thread(target=self.receive_camera_footage)
        self.receiving_footage_thread.start()
    else:
        self.switch_camera_request()
    except (BrokenPipeError, ConnectionResetError, socket.timeout,
OSError):
        self.app.show_exception_frame("Lost connection to server")

def receive_camera_footage(self):
    while self.receiving_footage:
        try:

client_socket.send(rsa_encrypt(constants.client_to_server_public_key,
"ready_for_pic").encode())

        received_data = b""
        payload_size = struct.calcsize("L")

        while len(received_data) < payload_size:
            received_data += client_socket.recv(4096)

        packed_msg_size = received_data[:payload_size]
        received_data = received_data[payload_size:]
        msg_size = struct.unpack("L", packed_msg_size)[0]

        while len(received_data) < msg_size:
            received_data += client_socket.recv(4096)

        frame_data = received_data[:msg_size]
        decrypted_frame_data =
aes_ecb_decrypt(self.camera_encryption_key, frame_data)

        nparr = numpy.frombuffer(decrypted_frame_data,
numpy.uint8)
        frame = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

        if self.mirrored:
            frame = numpy.flip(frame, axis=1)

```

```

        self.image_to_display_in_numpy_array = frame
    except socket.timeout:
        MessageBox(0, "Connection to camera lost", "Camera
Unavailable", MB_ICONERROR)
        self.image_to_display_in_numpy_array =
numpy.array(Image.open("no_video.jpg"))
        self.camera_title.configure(text="No camera - try to
switch cameras")
        self.receiving_footage = False
    except (BrokenPipeError, ConnectionResetError, OSError):
        self.app.show_exception_frame("Lost connection to
server")

    def show_camera_footage(self):
        captured_image =
Image.fromarray(self.image_to_display_in_numpy_array)
        photo_image = ImageTk.PhotoImage(image=captured_image)

        self.camera_label.photo_image = photo_image
        self.camera_label.configure(image=photo_image)

        self.camera_label.after(1, self.show_camera_footage)

class CameraSelectionFrame(CTkFrame):
    def __init__(self, app):
        super().__init__(app)
        self.app = app
        self.configure(fg_color="white")

        self.camera_active = False
        self.video_capture = None

        self.camera_index = CTkOptionMenu(self, values=list(map(lambda x:
f"Camera {x}", self.count_connected_cameras()))), width=250,
fg_color="white", button_color="gray29", button_hover_color="gray",
text_color="black")
        self.camera_index.pack(pady=(30, 10))

        self.test_camera_button = CTkButton(self, text="Test Selected
Camera", command=self.start_camera, font=("Consolas", 13, "bold"),
height=40, width=200, fg_color="black", hover_color="gray",
text_color="white")
        self.test_camera_button.pack(pady=10)

        self.camera_label = CTkLabel(self, text="")

```

```

self.camera_label.pack(pady=10)

self.select_camera_button = CTkButton(self, text="Select This
Camera", command=self.stop_camera, font=("Consolas", 13, "bold"),
height=40, width=200, fg_color="black", hover_color="gray",
text_color="white")
self.select_camera_button.pack(pady=(10, 30))

def update_camera_indexes(self):
    self.camera_index.configure(values =list(map(lambda x: f"Camera
{x}", self.count_connected_cameras()))))

def start_camera(self):
    self.camera_active = False
    time.sleep(0.1)
    camera_index = int(self.camera_index.get()[-2:])
    self.camera_active = True
    self.video_capture = cv2.VideoCapture(camera_index)
    width, height = 400, 300
    self.video_capture.set(cv2.CAP_PROP_FRAME_WIDTH, width)
    self.video_capture.set(cv2.CAP_PROP_FRAME_HEIGHT, height)

    threading.Thread(target=self.test_camera,
args=(camera_index,)).start()

def stop_camera(self):
    if self.camera_active:
        self.camera_active = False
        self.video_capture.release()
        self.app.show_camera_frame(int(self.camera_index.get()[-2:]))

def test_camera(self, camera_index):
    if not self.camera_active:
        return

    try:
        _, frame = self.video_capture.read()

        opencv_image = cv2.cvtColor(cv2.cvtColor(frame,
cv2.COLOR_BGR2GRAY), cv2.COLOR_BGR2RGBA)
        captured_image = Image.fromarray(opencv_image)
        photo_image = ImageTk.PhotoImage(image=captured_image)

        self.camera_label.photo_image = photo_image
        self.camera_label.configure(image=photo_image)

```

```

        self.camera_label.after(1, lambda:
self.test_camera(camera_index))
    except Exception as e:
        image = Image.open("no_video.jpg")
        photo_image = ImageTk.PhotoImage(image)
        self.camera_label.photo_image = photo_image
        self.camera_label.configure(image=photo_image)

def count_connected_cameras(self):
    camera_count = 0
    looking_for_cameras = True
    cameras_indexes = []

    while looking_for_cameras:
        cap = cv2.VideoCapture(camera_count)
        if cap.isOpened():
            ret, frame = cap.read()
            if ret:
                cameras_indexes.append(camera_count)
                cap.release()
            else:
                cap.release()
                looking_for_cameras = False

        camera_count += 1

    return cameras_indexes

class CameraFrame(CTkFrame):
    def __init__(self, app):
        super().__init__(app)
        self.app = app
        self.configure(fg_color="white")

        self.grid_columnconfigure(0, weight=1)
        self.grid_rowconfigure(0, weight=0)
        self.grid_rowconfigure(1, weight=1)
        self.grid_rowconfigure(2, weight=0)
        self.grid_rowconfigure(3, weight=0)

        self.encrypted_key = None
        self.video_capture = None

        self.camera_text_label = CTkLabel(self, text="Camera hidden for
you", text_color="black", font=("Arial", 14))

```

```

        self.camera_text_label.grid(row=0, column=0, padx=10, pady=(20,
5), sticky="n")

        self.camera_label = CTKLabel(self, text="")
        self.camera_label.grid(row=0, column=0, padx=10, pady=(0, 10),
sticky="n")

        self.showing_camera = False
        self.camera_loop_id = None

        self.show_camera_button = CTKButton(self, text="Show Camera
Footage", command=self.toggle_camera_footage, font=("Consolas", 13,
"bold"), height=40, width=200, fg_color="black", hover_color="gray",
text_color="white")
        self.show_camera_button.grid(row=1, column=0, padx=10, pady=(10,
5))

        self.mirrored = False
        self.mirror_button = CTKButton(self, text="Mirror Footage",
command=self.mirror_footage, font=("Consolas", 13, "bold"), height=40,
width=200, fg_color="black", hover_color="gray", text_color="white")
        self.mirror_button.grid(row=2, column=0, padx=10, pady=(5, 20))

    def start_sending_camera_footage(self, camera_index):
        self.encryption_key = os.urandom(16)

        try:
            client_socket.send(rsa_encrypt(constants.client_to_server_public_key,
self.encryption_key.hex()).encode())
        except (BrokenPipeError, ConnectionResetError, OSError):
            self.app.show_exception_frame("Lost connection to server")

        self.video_capture = cv2.VideoCapture(camera_index)
        width, height = 400, 300
        self.video_capture.set(cv2.CAP_PROP_FRAME_WIDTH, width)
        self.video_capture.set(cv2.CAP_PROP_FRAME_HEIGHT, height)

        threading.Thread(target=self.send_camera_footage).start()

    def mirror_footage(self):
        self.mirrored = not self.mirrored

    def toggle_camera_footage(self):
        if not self.showing_camera:
            self.showing_camera = True

```

```

        self.show_camera_button.configure(text="Hide camera footage")
        self.camera_label.grid(row=0, column=0, padx=10, pady=10) #
Return the label when showing video again
        self.show_camera_footage()
    else:
        self.showing_camera = False
        self.show_camera_button.configure(text="Show camera footage")
        if self.camera_loop_id is not None:
            self.after_cancel(self.camera_loop_id)
        # Clear the image from the label
        self.camera_label.configure(image=None)
        self.camera_label.photo_image = None
        self.camera_label.grid_remove() # Remove image label to show
text label

    def show_camera_footage(self):
        _, frame = self.video_capture.read()
        opencv_image = cv2.cvtColor(cv2.cvtColor(frame,
cv2.COLOR_BGR2GRAY), cv2.COLOR_BGR2RGBA)

        if self.mirrored:
            opencv_image = numpy.flip(opencv_image, axis=1)

        captured_image = Image.fromarray(opencv_image)
        photo_image = ImageTk.PhotoImage(image=captured_image)

        self.camera_label.photo_image = photo_image
        self.camera_label.configure(image=photo_image)

        if self.showing_camera:
            self.camera_loop_id = self.camera_label.after(1,
self.show_camera_footage)

    def send_camera_footage(self):
        try:
            while True:
                ret, frame = self.video_capture.read()
                frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY) # Turn
frame black and white

                if self.mirrored:
                    frame = numpy.flip(frame, axis=1)

                # Compress the frame using JPEG
                encode_param = [int(cv2.IMWRITE_JPEG_QUALITY), 30]
                _, encoded_frame = cv2.imencode('.jpg', frame,

```

```

encode_param)

        frame_bytes = encoded_frame.tobytes()
        encrypted_frame_bytes =
aes_ecb_encrypt(self.encryption_key, frame_bytes)

        message_size = struct.pack("L",
len(encrypted_frame_bytes))
        client_socket.sendall(message_size +
encrypted_frame_bytes)

        # Image received confirmation -> send next one
        client_socket.recv(1024)
    except (BrokenPipeError, ConnectionResetError, socket.timeout,
OSError):
        self.app.show_exception_frame("Lost connection to server")

if __name__ == "__main__":
    try:
        client_socket = socket.socket()
        client_socket.settimeout(5)
        client_socket.connect(("127.0.0.1", 1729))

        server_response = client_socket.recv(1024).decode()
        if server_response == "Server full":
            MessageBox(0, "Disconnected: Server is full", "Connection
Failed", MB_ICONERROR)
            client_socket.close()
        elif server_response == "blacklisted":
            MessageBox(0, "Blacklisted: Too many failed login attempts",
"Connection Denied", MB_ICONERROR)
            client_socket.close()
        elif server_response == "Connected to server":
            root = App()
            root.mainloop()
    except (ConnectionRefusedError, socket.timeout, ConnectionResetError,
BrokenPipeError, OSError):
        MessageBox(0, "Could not connect to the server", "Server
Unavailable", MB_ICONERROR)

```

קוד השרת:

```

import socket, select
import sqlite3
import threading

```

```
import constants
import db
import struct
from encryption import *
import telegram_bot

class Client:
    def __init__(self, client_socket, name):
        self.client_socket = client_socket
        self.name = name

class User(Client):
    def __init__(self, client_socket, username, user_id, chat_id):
        super().__init__(client_socket, username)
        self.user_id = user_id
        self.current_camera_id = None
        self.chat_id = chat_id
        self.ready_for_pic = False

class Camera(Client):
    def __init__(self, client_socket, camera_name, camera_id):
        super().__init__(client_socket, camera_name)
        self.camera_id = camera_id
        self.encryption_key = None

def handle_client(client_socket, ip_addr, login_attempts_counter=0):
    try:
        msg = client_socket.recv(1024)
        if not msg:
            raise ConnectionResetError("Client disconnected")

        message = rsa_decrypt(constants.client_to_server_private_key,
msg.decode())

        if message == "register":

client_socket.send(rsa_encrypt(constants.server_to_client_public_key,
"Register attempt detected").encode())
        msg = client_socket.recv(1024)
        if not msg:
            raise ConnectionResetError("Client disconnected")
        register_info =
```

```
rsa_decrypt(constants.client_to_server_private_key,
msg.decode()).split() # 0 - username, 1 - password, 2 - account_type

    try:
        db_lock.acquire()
        db.create_new_account(register_info[0], register_info[1])
        db_lock.release()

        if register_info[2] == "User":

client_socket.send(rsa_encrypt(constants.server_to_client_public_key,
"verification code needed").encode())
        getting_code = True
        chat_id = ""
        while getting_code:
            msg = client_socket.recv(1024)
            if not msg:
                raise ConnectionResetError("Client
disconnected")

            verif_code =
rsa_decrypt(constants.client_to_server_private_key, msg.decode())
            try:
                chat_id =
db.get_chat_id_from_verif_code(verif_code)
                getting_code = False
            except TypeError:

client_socket.send(rsa_encrypt(constants.server_to_client_public_key,
"No such verification code").encode())

                db_lock.acquire()
                db.create_new_user(register_info[0], chat_id)
                db_lock.release()

client_socket.send(rsa_encrypt(constants.server_to_client_public_key,
"User account created successfully").encode())

                user = User(client_socket, register_info[0],
db.get_user_id(register_info[0]), chat_id)
                users.append(user)
                handle_user(user)

        elif register_info[2] == "Camera":
            db_lock.acquire()
            db.create_new_camera(register_info[0])
            db_lock.release()
```

```

client_socket.send(rsa_encrypt(constants.server_to_client_public_key,
"Camera account created successfully").encode())

        camera = Camera(client_socket, register_info[0],
db.get_camera_id(register_info[0]))
        cameras.append(camera)
        handle_camera(camera)

    except sqlite3.IntegrityError:

client_socket.send(rsa_encrypt(constants.server_to_client_public_key,
"Attempted username is taken").encode())
        db_lock.release()
        handle_client(client_socket, ip_addr,
login_attempts_counter)

    elif message == "Login":

client_socket.send(rsa_encrypt(constants.server_to_client_public_key,
>Login attempt detected").encode())
        msg = client_socket.recv(1024)
        if not msg:
            raise ConnectionResetError("Client disconnected")
        login_info =
rsa_decrypt(constants.client_to_server_private_key,
msg.decode()).split()

        try:
            db_lock.acquire()
            db.Login(login_info[0], login_info[1])
            account_type = db.get_account_type(login_info[0])
            db_lock.release()

            if account_type == "User":
                user = User(client_socket, login_info[0],
db.get_user_id(login_info[0]), db.get_chat_id(login_info[0]))
                for u in users:
                    if u.user_id == user.user_id:
                        db_lock.acquire()
                        raise Exception("User already logged in")

client_socket.send(rsa_encrypt(constants.server_to_client_public_key,
>User logged in successfully").encode())
                users.append(user)
                handle_user(user)

```

```

        elif account_type == "Camera":
            camera = Camera(client_socket, login_info[0],
db.get_camera_id(login_info[0]))
            for cam in cameras:
                if cam.camera_id == camera.camera_id:
                    db_lock.acquire()
                    raise Exception("Camera already logged in")

client_socket.send(rsa_encrypt(constants.server_to_client_public_key,
"Camera logged in successfully").encode())
            cameras.append(camera)
            handle_camera(camera)

    except Exception as e:
        login_attempts_counter += 1
        if login_attempts_counter >= 3:
            blacklisted_ips.add(ip_addr)

client_socket.send(rsa_encrypt(constants.server_to_client_public_key,
"Too many login attempts").encode())
        else:

client_socket.send(rsa_encrypt(constants.server_to_client_public_key,
f"{e}").encode())
            db_lock.release()
            handle_client(client_socket, ip_addr,
login_attempts_counter)
    except Exception as e:
        if client_socket in clients:
            clients.remove(client_socket)
            client_socket.close()

def handle_user(user):
    user.ready_for_pic = True
    try:
        while True:
            msg = user.client_socket.recv(1024)
            if not msg:
                raise ConnectionResetError("Client disconnected")

            request = rsa_decrypt(constants.client_to_server_private_key,
msg.decode())

            if "ready_for_pic" in request:

```

```

        user.ready_for_pic = True

    elif "subscribe" in request and user.current_camera_id is not
None:
        try:
            camera_sub_lock.acquire()
            db.subscribe_to_camera(user.name,
user.current_camera_id)
        finally:
            camera_sub_lock.release()

    elif "alert" in request and user.current_camera_id is not
None:
        try:
            camera_sub_lock.acquire()
            subs =
db.get_camera_subscriptions(user.current_camera_id)
        finally:
            camera_sub_lock.release()
            threading.Thread(target=send_alert_messages, args=(subs,
user.name, user.current_camera_id)).start()

    elif "switch_camera_request" in request:
        switch_camera(user)
        user.ready_for_pic = True
except Exception as e:
    if user in users:
        users.remove(user)
    if user.client_socket in clients:
        clients.remove(user.client_socket)
    user.client_socket.close()

def switch_camera(user):
    curr_camera = False
    for camera in cameras:
        if camera.camera_id == user.current_camera_id:
            curr_camera = True
            index = cameras.index(camera)
            if len(cameras) - 1 > index:
                user.current_camera_id = cameras[index + 1].camera_id
                get_camera_info(user)
            elif len(cameras) - 1 == index:
                user.current_camera_id = cameras[0].camera_id
                get_camera_info(user)
    break

```

```

if not curr_camera:
    try:
        user.current_camera_id = cameras[0].camera_id
        get_camera_info(user)
    except IndexError:
        user.current_camera_id = None

user.client_socket.send(rsa_encrypt(constants.server_to_client_public_key, "no_cameras").encode())

def get_camera_info(user):
    for cam in cameras:
        if user.current_camera_id == cam.camera_id:
            try:
                camera_sub_lock.acquire()
                subs =
db.get_camera_subscriptions(user.current_camera_id)
            finally:
                camera_sub_lock.release()

user.client_socket.send(rsa_encrypt(constants.server_to_client_public_key, f"{cam.name} {cam.camera_id} {user.name in subs}").encode())
ans = user.client_socket.recv(1024)
if not ans:
    raise ConnectionResetError("Client disconnected")

user.client_socket.sendall(rsa_encrypt(constants.server_to_client_public_key, cam.encrypted_key).encode())
break

def send_alert_messages(subs, alerter_name, camera_id):
    for sub in subs:
        if sub != alerter_name:
            chat_id = int(db.get_chat_id(sub))
            try:
                telegram_bot.send_telegram_message(chat_id, f"User {alerter_name} has decided to alert all users of the camera with the id of {camera_id} ")
            except Exception as e:
                pass

def handle_camera(camera):

```

```

try:
    encryption_key = camera.client_socket.recv(1024)
    if not encryption_key:
        raise ConnectionResetError("Camera disconnected")

    camera.encryption_key =
rsa_decrypt(constants.client_to_server_private_key,
encryption_key.decode())
    camera.client_socket.settimeout(10)
    while True:
        received_data = b""
        payload_size = struct.calcsize("L")

        while len(received_data) < payload_size:
            try:
                data = camera.client_socket.recv(4096)
            except socket.timeout:
                raise TimeoutError("Camera timeout")
            if not data:
                raise ConnectionResetError("Camera disconnected")
            received_data += data

        packed_msg_size = received_data[:payload_size]
        received_data = received_data[payload_size:]
        msg_size = struct.unpack("L", packed_msg_size)[0]

        while len(received_data) < msg_size:
            try:
                data = camera.client_socket.recv(4096)
            except socket.timeout:
                raise TimeoutError("Camera timeout")
            if not data:
                raise ConnectionResetError("Camera disconnected")
            received_data += data

        frame_data = received_data[:msg_size]
        try:
            camera.client_socket.send("fir".encode()) # full image
received
        except Exception as e:
            raise ConnectionResetError(f"Failed to send message to
camera {e}")

        for user in users:
            if user.current_camera_id == camera.camera_id and
user.ready_for_pic:

```

```
        try:
            user.ready_for_pic = False
            message_size = struct.pack("L", len(frame_data))
            user.client_socket.sendall(message_size +
frame_data)

        except Exception:
            pass

    except Exception as e:
        if camera in cameras:
            for user in users:
                if user.current_camera_id == camera.camera_id:
                    user.current_camera_id = None
                    user.ready_for_pic = False
            cameras.remove(camera)
        if camera.client_socket in clients:
            clients.remove(camera.client_socket)
            camera.client_socket.close()

if __name__ == "__main__":
    db.main()

    server = socket.socket()
    server.bind(('0.0.0.0', 1729))
    max_clients_num = 10
    server.listen(max_clients_num)
    clients = []
    users = []
    cameras = []
    db_lock = threading.Lock()
    camera_sub_lock = threading.Lock()
    blacklisted_ips = set()

    while True:
        try:
            rlist, wlist, xlist = select.select([server] + clients,
clients, clients)

            for client in xlist:
                if client in clients:
                    clients.remove(client)
                    client.close()

            for client in rlist:
                if client is server:
```

```

c, address = client.accept()
if len(clients) >= max_clients_num:
    c.sendall("Server full".encode())
    c.close()
elif address[0] in blacklisted_ips:
    c.sendall("blacklisted".encode())
    c.close()
else:
    c.sendall("Connected to server".encode())
    clients.append(c)
    threading.Thread(target=handle_client,
args=(c, address[0])).start()
except (OSError, BrokenPipeError):
    pass

```

קוד בסיס הנתונים:

```

import sqlite3
import hashlib
from encryption import *
import constants
import functools

def main():
    conn = sqlite3.connect("watchdog.db")
    conn.execute("PRAGMA foreign_keys = ON;")

    # create Account table
    conn.execute(''' CREATE TABLE IF NOT EXISTS Account
        ( name TEXT NOT NULL UNIQUE PRIMARY KEY,
          password TEXT NOT NULL) ''')

    # create User table
    conn.execute(''' CREATE TABLE IF NOT EXISTS User
        ( id INTEGER PRIMARY KEY AUTOINCREMENT,
          name TEXT NOT NULL,
          chat_id TEXT NOT NULL,
          FOREIGN KEY(name) REFERENCES Account(name)) ''')

    # create Camera table
    conn.execute(''' CREATE TABLE IF NOT EXISTS Camera
        ( id INTEGER PRIMARY KEY AUTOINCREMENT,
          name TEXT NOT NULL,
          linked_users TEXT NOT NULL,
          FOREIGN KEY(name) REFERENCES Account(name)) ''')

```

```
# create / delete and create Telegram chat ID to verification code
table
conn.execute('''DROP TABLE IF EXISTS ChatIDs''')
conn.execute(''' CREATE TABLE IF NOT EXISTS ChatIDs
                ( chat_id TEXT NOT NULL,
                  verif_code TEXT NOT NULL ) ''')

conn.close()

def create_new_account(name, password):
    conn = sqlite3.connect("watchdog.db")
    result = None

    try:
        conn.execute('''
                        INSERT INTO Account (name, password)
                        VALUES(?,?)
                        ''',
[rsa_encrypt(constants.server_to_db_public_key, name),
hash_password(password)])
    except sqlite3.IntegrityError as e:
        result = e

    conn.commit()
    conn.close()

    if result is not None:
        raise result

def create_new_user(name, chat_id):
    conn = sqlite3.connect("watchdog.db")

    conn.execute("INSERT INTO User (name, chat_id) VALUES (?, ?)",
[rsa_encrypt(constants.server_to_db_public_key, name),
rsa_encrypt(constants.server_to_db_public_key, chat_id)])

    conn.commit()
    conn.close()

def create_new_camera(name):
    conn = sqlite3.connect("watchdog.db")
```

```
conn.execute("INSERT INTO Camera (name, linked_users) VALUES (?, ?)",
             [rsa_encrypt(constants.server_to_db_public_key, name),
              ""])

conn.commit()
conn.close()

def create_new_chat_id_verif_code(chat_id, verif_code):
    conn = sqlite3.connect("watchdog.db")

    conn.execute("INSERT INTO ChatIDs (chat_id, verif_code) VALUES (?,
    ?)",
                 [rsa_encrypt(constants.server_to_db_public_key,
chat_id), rsa_encrypt(constants.server_to_db_public_key, verif_code)])

    conn.commit()
    conn.close()

def login(name, password):
    result = None
    conn = sqlite3.connect("watchdog.db")
    cursor = conn.cursor()

    cursor.execute('SELECT * FROM Account WHERE name = ?',
(rsa_encrypt(constants.server_to_db_public_key, name),))
    account = cursor.fetchone()

    if account:
        if account[1] == hash_password(password):
            pass # Password matches (hashed password matches hashed
password)
        else:
            result = Exception("Wrong password")
    else:
        result = Exception("No such username")

    conn.close()

    if result is not None:
        raise result

def get_chat_id_from_verif_code(verif_code):
    conn = sqlite3.connect("watchdog.db")
```

```
cursor = conn.cursor()

cursor.execute('SELECT * FROM ChatIDs WHERE verif_code = ?',
(rsa_encrypt(constants.server_to_db_public_key, verif_code),))
chat_id = rsa_decrypt(constants.server_to_db_private_key,
cursor.fetchone()[0])

cursor.execute('DELETE FROM ChatIDs WHERE verif_code = ?',
(rsa_encrypt(constants.server_to_db_public_key, verif_code),))
conn.commit()

conn.close()
return chat_id

def get_chat_id(name):
    conn = sqlite3.connect("watchdog.db")
    cursor = conn.cursor()

    cursor.execute('SELECT * FROM User WHERE name = ?',
(rsa_encrypt(constants.server_to_db_public_key, name),))
    chat_id = rsa_decrypt(constants.server_to_db_private_key,
cursor.fetchone()[2])

    conn.close()
    return chat_id

def get_user_id(name):
    conn = sqlite3.connect("watchdog.db")
    cursor = conn.cursor()

    cursor.execute('SELECT * FROM User WHERE name = ?',
(rsa_encrypt(constants.server_to_db_public_key, name),))
    uid = cursor.fetchone()[0]

    conn.close()
    return uid

def get_camera_id(name):
    conn = sqlite3.connect("watchdog.db")
    cursor = conn.cursor()

    cursor.execute('SELECT * FROM Camera WHERE name = ?',
(rsa_encrypt(constants.server_to_db_public_key, name),))
    cid = cursor.fetchone()[0]
```

```
conn.close()
return cid

def get_account_type(name):
    conn = sqlite3.connect("watchdog.db")
    cursor = conn.cursor()
    result = ""

    cursor.execute('SELECT * FROM User WHERE name = ?',
(rsa_encrypt(constants.server_to_db_public_key, name),))
    info = cursor.fetchone()

    if info is not None:
        result = "User"
    else:
        result = "Camera"

    conn.close()
    return result

def get_camera_subscriptions(cid):
    conn = sqlite3.connect("watchdog.db")
    cursor = conn.cursor()

    cursor.execute('SELECT * FROM Camera WHERE id = ?', (cid,))
    linked_users = cursor.fetchone()[2]
    if linked_users == '':
        subs = []
    else:
        subs = rsa_decrypt(constants.server_to_db_private_key,
linked_users).split(',')

    conn.close()
    return subs

def subscribe_to_camera(username, cid):
    subs = get_camera_subscriptions(cid)
    subs_to_db = ""
    print(subs)
    if username not in subs:
        subs.append(username)
        subs_to_db = functools.reduce(lambda x, y: f"{x},{y}", subs)
```

```

        subs_to_db = rsa_encrypt(constants.server_to_db_public_key,
subs_to_db)
        print(subs_to_db)

        conn = sqlite3.connect("watchdog.db")
        cursor = conn.cursor()

        cursor.execute('UPDATE Camera SET linked_users = ? WHERE id = ?',
(subs_to_db, cid))
        conn.commit()
        conn.close()

def hash_password(password):
    password_hash = hashlib.sha256(password.encode()).hexdigest()
    return password_hash

if __name__ == "__main__":
    main()

```

קוד הצפנה:

```

import functools
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms,
modes
from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives import padding

# RSA
def rsa_encrypt(pk, plaintext):
    key, n = pk
    # a^b mod m
    cipher = [(ord(char) ** key) % n for char in plaintext]
    return functools.reduce(lambda x, y: f"{x},{y}", cipher)

def rsa_decrypt(pk, ciphertext):
    key, n = pk
    # a^b mod m
    plain = [chr((char ** key) % n) for char in list(map(lambda x:
int(x), ciphertext.split(",")))]
    return "".join(plain)

# AES
def aes_ecb_encrypt(key, data):

```

```

padder = padding.PKCS7(128).padder()
padded_data = padder.update(data) + padder.finalize()

cipher = Cipher(algorithms.AES(key), modes.ECB(),
backend=default_backend())
encryptor = cipher.encryptor()
return encryptor.update(padded_data) + encryptor.finalize()

def aes_ecb_decrypt(key, encrypted_data):
    cipher = Cipher(algorithms.AES(key), modes.ECB(),
backend=default_backend())
    decryptor = cipher.decryptor()
    padded_data = decryptor.update(encrypted_data) + decryptor.finalize()

    unpadder = padding.PKCS7(128).unpadder()
    data = unpadder.update(padded_data) + unpadder.finalize()
    return data

```

קוד בוט טלגרם:

```

from telegram import Update
from telegram.ext import Application, CommandHandler, ContextTypes
import random
import constants
import db
import requests

BOT_TOKEN = constants.TELEGRAM_BOT_TOKEN

def send_telegram_message(chat_id, message):
    url = f"https://api.telegram.org/bot{BOT_TOKEN}/sendMessage"
    payload = {"chat_id": chat_id, "text": message} # json

    try:
        response = requests.post(url, data=payload)
        response.raise_for_status()
        return True
    except requests.RequestException as e:
        return False

def generate_six_digit_code():
    return random.randint(100000, 999999)

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    chat_id = update.message.chat.id
    verif_code = generate_six_digit_code()

```

```
await context.bot.send_message(chat_id=chat_id, text=f"Your
verification code is: {verif_code}")

db.create_new_chat_id_verif_code(str(chat_id), str(verif_code))

def main():
    application = Application.builder().token(BOT_TOKEN).build()
    application.add_handler(CommandHandler("start", start))
    application.run_polling()

if __name__ == '__main__':
    main()
```

קוד קבועים:

```
TELEGRAM_BOT_TOKEN = 'Hidden for security purposes'

# client -> server keys
client_to_server_public_key = (1253, 3071)
client_to_server_private_key = (2813, 3071)

# server -> client keys
server_to_client_public_key = (279, 943)
server_to_client_private_key = (839, 943)

# server -> db keys
server_to_db_public_key = (4813, 8633)
(server_to_db_private_key = (8197, 8633
```